```c
/*
 * serial_comm.c — cross-platform serial port
 */
#include "serial_comm.h"
#include <stdio.h>
#include <string.h>

/* ================================================================
 *  Platform: Windows — CreateFile on COMx
 * ================================================================ */
#ifdef _WIN32

#include <windows.h>

static HANDLE g_hcom = INVALID_HANDLE_VALUE;

bool serial_open(const char *port, int baud, int data_bits,
                 char parity, int stop_bits)
{
    char path[32];
    DCB dcb = {0};
    COMMTIMEOUTS to = {0};

    serial_close();

    snprintf(path, sizeof(path), "\\\\.\\%s", port);
    g_hcom = CreateFileA(path, GENERIC_READ | GENERIC_WRITE,
                         0, NULL, OPEN_EXISTING, 0, NULL);
    if (g_hcom == INVALID_HANDLE_VALUE) {
        fprintf(stderr, "[SERIAL] Cannot open %s\n", port);
        return false;
    }

    dcb.DCBlength = sizeof(dcb);
    GetCommState(g_hcom, &dcb);
    dcb.BaudRate   = baud;
    dcb.ByteSize   = (BYTE)data_bits;
    dcb.Parity     = (parity == 'N') ? NOPARITY :
                     (parity == 'E') ? EVENPARITY :
                     (parity == 'O') ? ODDPARITY  : NOPARITY;
    dcb.StopBits   = (stop_bits == 2) ? TWOSTOPBITS : ONESTOPBIT;
    SetCommState(g_hcom, &dcb);

    to.ReadIntervalTimeout         = 10;
    to.ReadTotalTimeoutMultiplier  = 0;
    to.ReadTotalTimeoutConstant    = 50;
    SetCommTimeouts(g_hcom, &to);

    printf("[SERIAL] Opened %s %d-%d%c%d\n", port, baud, data_bits, parity, stop_bits);
    return true;
}

int serial_send(const uint8_t *data, int len)
{
    if (g_hcom == INVALID_HANDLE_VALUE) return -1;
    DWORD written = 0;
    WriteFile(g_hcom, data, (DWORD)len, &written, NULL);
    return (int)written;
}

int serial_recv(uint8_t *buf, int max_len)
{
    if (g_hcom == INVALID_HANDLE_VALUE) return -1;
```

```
    DWORD n = 0;
    ReadFile(g_hcom, buf, (DWORD)max_len, &n, NULL);
    return (int)n;
}

void serial_close(void)
{
    if (g_hcom != INVALID_HANDLE_VALUE) {
        CloseHandle(g_hcom);
        g_hcom = INVALID_HANDLE_VALUE;
        printf("[SERIAL] Closed\n");
    }
}

bool serial_is_open(void)
{
    return (g_hcom != INVALID_HANDLE_VALUE);
}

/* ============================================================
 *  Platform: D133CBS — RT-Thread device framework
 * ============================================================ */
#elif defined(__RTTHREAD__) || defined(RT_USING_LWIP)

#include <rtthread.h>
#include <rtdevice.h>
#include <stdint.h>

/* SDK internal — aic_uart_configure writes baud/func/parity to HW.
 * Not in any public header; use uint32_t to avoid u32 dependency. */
extern rt_err_t aic_uart_configure(uint32_t idx,
                                   struct serial_configure *cfg);

static rt_device_t g_dev = NULL;

bool serial_open(const char *port, int baud, int data_bits,
                 char parity, int stop_bits)
{
    struct serial_configure cfg = RT_SERIAL_CONFIG_DEFAULT;

    serial_close();

    g_dev = rt_device_find(port);
    if (!g_dev) {
        printf("[SERIAL] Device %s not found\n", port);
        return false;
    }

    cfg.baud_rate = baud;
    cfg.data_bits = (data_bits == 8) ? DATA_BITS_8 : DATA_BITS_7;
    cfg.stop_bits = (stop_bits == 2) ? STOP_BITS_2 : STOP_BITS_1;
    if (parity == 'E')      cfg.parity = PARITY_EVEN;
    else if (parity == 'O') cfg.parity = PARITY_ODD;
    else                    cfg.parity = PARITY_NONE;

    /* RS485 COMPACT_IO: PD.2=TX/RX, PD.3=DE(RTS muxed to RX pin) */
    cfg.function = 2;  /* USART_FUNC_RS485_COMACT_IO */

    /* rt_device_control(RT_DEVICE_CTRL_CONFIG) is NOT handled by this
     * driver; use SDK's own aic_uart_configure to write HW registers */
    aic_uart_configure(1, &cfg);
    rt_device_open(g_dev, RT_DEVICE_FLAG_RDWR | RT_DEVICE_FLAG_INT_RX);
```

```
    printf("[SERIAL] Opened %s %d-%d%c%d\n", port, baud, data_bits, parity, stop_bits);
    return true;
}

int serial_send(const uint8_t *data, int len)
{
    if (!g_dev) return -1;
    int ret = rt_device_write(g_dev, 0, data, len);
    if (ret <= 0) printf("[SERIAL] write returned %d\n", ret);
    return ret;
}

int serial_recv(uint8_t *buf, int max_len)
{
    if (!g_dev) return -1;
    int ret = rt_device_read(g_dev, 0, buf, max_len);
    if (ret < 0) return ret;
    return ret;
}

void serial_close(void)
{
    if (g_dev) {
        rt_device_close(g_dev);
        g_dev = NULL;
        printf("[SERIAL] Closed\n");
    }
}

bool serial_is_open(void) { return (g_dev != NULL); }

#else
    /* stub */
    bool serial_open(const char *p, int b, int d, char pa, int s)
    { (void)p;(void)b;(void)d;(void)pa;(void)s; return false; }
    int  serial_send(const uint8_t *d, int l) { (void)d;(void)l; return -1; }
    int  serial_recv(uint8_t *b, int m) { (void)b;(void)m; return -1; }
    void serial_close(void) {}
    bool serial_is_open(void) { return false; }
#endif

/*
 * modbus_rtu.c — Modbus RTU frame builder + CRC16
 */
#include "modbus_rtu.h"
#include <stdio.h>
#include <string.h>

/* ---- CRC16 lookup table (polynomial 0xA001) ---- */
static const uint16_t crc_table[256] = {
    0x0000,0xC0C1,0xC181,0x0140,0xC301,0x03C0,0x0280,0xC241,
    0xC601,0x06C0,0x0780,0xC741,0x0500,0xC5C1,0xC481,0x0440,
    0xCC01,0x0CC0,0x0D80,0xCD41,0x0F00,0xCFC1,0xCE81,0x0E40,
    0x0A00,0xCAC1,0xCB81,0x0B40,0xC901,0x09C0,0x0880,0xC841,
    0xD801,0x18C0,0x1980,0xD941,0x1B00,0xDBC1,0xDA81,0x1A40,
    0x1E00,0xDEC1,0xDF81,0x1F40,0xDD01,0x1DC0,0x1C80,0xDC41,
    0x1400,0xD4C1,0xD581,0x1540,0xD701,0x17C0,0x1680,0xD641,
    0xD201,0x12C0,0x1380,0xD341,0x1100,0xD1C1,0xD081,0x1040,
    0xF001,0x30C0,0x3180,0xF141,0x3300,0xF3C1,0xF281,0x3240,
    0x3600,0xF6C1,0xF781,0x3740,0xF501,0x35C0,0x3480,0xF441,
    0x3C00,0xFCC1,0xFD81,0x3D40,0xFF01,0x3FC0,0x3E80,0xFE41,
    0xFA01,0x3AC0,0x3B80,0xFB41,0x3900,0xF9C1,0xF881,0x3840,
    0x2800,0xE8C1,0xE981,0x2940,0xEB01,0x2BC0,0x2A80,0xEA41,
```

```
    0xEE01, 0x2EC0, 0x2F80, 0xEF41, 0x2D00, 0xEDC1, 0xEC81, 0x2C40,
    0xE401, 0x24C0, 0x2580, 0xE541, 0x2700, 0xE7C1, 0xE681, 0x2640,
    0x2200, 0xE2C1, 0xE381, 0x2340, 0xE101, 0x21C0, 0x2080, 0xE041,
    0xA001, 0x60C0, 0x6180, 0xA141, 0x6300, 0xA3C1, 0xA281, 0x6240,
    0x6600, 0xA6C1, 0xA781, 0x6740, 0xA501, 0x65C0, 0x6480, 0xA441,
    0x6C00, 0xACC1, 0xAD81, 0x6D40, 0xAF01, 0x6FC0, 0x6E80, 0xAE41,
    0xAA01, 0x6AC0, 0x6B80, 0xAB41, 0x6900, 0xA9C1, 0xA881, 0x6840,
    0x7800, 0xB8C1, 0xB981, 0x7940, 0xBB01, 0x7BC0, 0x7A80, 0xBA41,
    0xBE01, 0x7EC0, 0x7F80, 0xBF41, 0x7D00, 0xBDC1, 0xBC81, 0x7C40,
    0xB401, 0x74C0, 0x7580, 0xB541, 0x7700, 0xB7C1, 0xB681, 0x7640,
    0x7200, 0xB2C1, 0xB381, 0x7340, 0xB101, 0x71C0, 0x7080, 0xB041,
    0x5000, 0x90C1, 0x9181, 0x5140, 0x9301, 0x53C0, 0x5280, 0x9241,
    0x9601, 0x56C0, 0x5780, 0x9741, 0x5500, 0x95C1, 0x9481, 0x5440,
    0x9C01, 0x5CC0, 0x5D80, 0x9D41, 0x5F00, 0x9FC1, 0x9E81, 0x5E40,
    0x5A00, 0x9AC1, 0x9B81, 0x5B40, 0x9901, 0x59C0, 0x5880, 0x9841,
    0x8801, 0x48C0, 0x4980, 0x8941, 0x4B00, 0x8BC1, 0x8A81, 0x4A40,
    0x4E00, 0x8EC1, 0x8F81, 0x4F40, 0x8D01, 0x4DC0, 0x4C80, 0x8C41,
    0x4400, 0x84C1, 0x8581, 0x4540, 0x8701, 0x47C0, 0x4680, 0x8641,
    0x8201, 0x42C0, 0x4380, 0x8341, 0x4100, 0x81C1, 0x8081, 0x4040,
};

uint16_t modbus_crc16(const uint8_t *data, int len)
{
    uint16_t crc = 0xFFFF;
    for (int i = 0; i < len; i++)
        crc = (crc >> 8) ^ crc_table[(crc ^ data[i]) & 0xFF];
    return crc;
}

uint16_t modbus_param_to_reg(int param_idx, bool non_standard)
{
    if (param_idx < 0 || param_idx >= 19) return 0;
    if (non_standard)
        return (uint16_t)param_idx;
    else {
        static const uint16_t table[19] = {
            REG_SP00, REG_SP01, REG_SP02, REG_SP03, REG_SP04,
            REG_SP05, REG_SP06, REG_SP07, REG_SP08, REG_SP09,
            REG_SP10, REG_SP11, REG_SP12, REG_SP13, REG_SP14,
            REG_SP15, REG_SP16, REG_SP17, REG_SP18,
        };
        return table[param_idx];
    }
}

int modbus_write_single(uint8_t slave, uint16_t reg, uint16_t value,
                        uint8_t *out, bool non_standard)
{
    (void)non_standard;
    out[0] = slave;
    out[1] = 0x06;
    out[2] = (uint8_t)(reg >> 8);
    out[3] = (uint8_t)(reg & 0xFF);
    out[4] = (uint8_t)(value >> 8);
    out[5] = (uint8_t)(value & 0xFF);

    uint16_t crc = modbus_crc16(out, 6);
    out[6] = (uint8_t)(crc & 0xFF);
    out[7] = (uint8_t)(crc >> 8);

    return 8;
}
```

```c
int modbus_write_multiple(uint8_t slave, uint16_t start,
                          const uint16_t *vals, int count,
                          uint8_t *out, int max_len, bool non_standard)
{
    (void)non_standard;
    int byte_count = count * 2;
    int total = 9 + byte_count;
    if (total > max_len) return -1;

    out[0] = slave;
    out[1] = 0x10;
    out[2] = (uint8_t)(start >> 8);
    out[3] = (uint8_t)(start & 0xFF);
    out[4] = (uint8_t)(count >> 8);
    out[5] = (uint8_t)(count & 0xFF);
    out[6] = (uint8_t)byte_count;

    for (int i = 0; i < count; i++) {
        out[7 + i * 2]     = (uint8_t)(vals[i] >> 8);
        out[7 + i * 2 + 1] = (uint8_t)(vals[i] & 0xFF);
    }

    uint16_t crc = modbus_crc16(out, total - 2);
    out[total - 2] = (uint8_t)(crc & 0xFF);
    out[total - 1] = (uint8_t)(crc >> 8);

    return total;
}

int modbus_read_holding(uint8_t slave, uint16_t start, uint16_t count,
                        uint8_t *out, bool non_standard)
{
    (void)non_standard;
    out[0] = slave;
    out[1] = 0x03;
    out[2] = (uint8_t)(start >> 8);
    out[3] = (uint8_t)(start & 0xFF);
    out[4] = (uint8_t)(count >> 8);
    out[5] = (uint8_t)(count & 0xFF);

    uint16_t crc = modbus_crc16(out, 6);
    out[6] = (uint8_t)(crc & 0xFF);
    out[7] = (uint8_t)(crc >> 8);

    return 8;
}

/* ─── 0x04 Read Input Registers ──────────────────────────────────────────
─── */
int modbus_read_input_regs(uint8_t slave, uint16_t start, uint16_t count,
                           uint8_t *out, bool non_standard)
{
    (void)non_standard;
    out[0] = slave;
    out[1] = 0x04;
    out[2] = (uint8_t)(start >> 8);
    out[3] = (uint8_t)(start & 0xFF);
    out[4] = (uint8_t)(count >> 8);
    out[5] = (uint8_t)(count & 0xFF);
    uint16_t crc = modbus_crc16(out, 6);
    out[6] = (uint8_t)(crc & 0xFF);
    out[7] = (uint8_t)(crc >> 8);
    return 8;
```

```
}

/* ─── 0x01 Read Coils ──────────────────────────────────────────────────────── */
int modbus_read_coils(uint8_t slave, uint16_t addr, uint16_t count,
                      uint8_t *out, bool non_standard)
{
    (void)non_standard;
    out[0] = slave;
    out[1] = 0x01;
    out[2] = (uint8_t)(addr >> 8);
    out[3] = (uint8_t)(addr & 0xFF);
    out[4] = (uint8_t)(count >> 8);
    out[5] = (uint8_t)(count & 0xFF);
    uint16_t crc = modbus_crc16(out, 6);
    out[6] = (uint8_t)(crc & 0xFF);
    out[7] = (uint8_t)(crc >> 8);
    return 8;
}

/* ─── 0x02 Read Discrete Inputs ────────────────────────────────────────────── */
int modbus_read_discrete(uint8_t slave, uint16_t addr, uint16_t count,
                         uint8_t *out, bool non_standard)
{
    (void)non_standard;
    out[0] = slave;
    out[1] = 0x02;
    out[2] = (uint8_t)(addr >> 8);
    out[3] = (uint8_t)(addr & 0xFF);
    out[4] = (uint8_t)(count >> 8);
    out[5] = (uint8_t)(count & 0xFF);
    uint16_t crc = modbus_crc16(out, 6);
    out[6] = (uint8_t)(crc & 0xFF);
    out[7] = (uint8_t)(crc >> 8);
    return 8;
}

/* ─── 0x05 Write Single Coil ───────────────────────────────────────────────── */
int modbus_write_coil(uint8_t slave, uint16_t addr, bool on,
                      uint8_t *out, bool non_standard)
{
    (void)non_standard;
    out[0] = slave;
    out[1] = 0x05;
    out[2] = (uint8_t)(addr >> 8);
    out[3] = (uint8_t)(addr & 0xFF);
    out[4] = on ? 0xFF : 0x00;
    out[5] = on ? 0x00 : 0x00;
    uint16_t crc = modbus_crc16(out, 6);
    out[6] = (uint8_t)(crc & 0xFF);
    out[7] = (uint8_t)(crc >> 8);
    return 8;
}

/* ─── 0x0F Write Multiple Coils ────────────────────────────────────────────── */
int modbus_write_coils(uint8_t slave, uint16_t addr,
                       const uint8_t *data, uint16_t count,
                       uint8_t *out, int max_len, bool non_standard)
{
    (void)non_standard;
```

```
    int byte_count = (count + 7) / 8;  /* ceil(count/8) */
    int total = 9 + byte_count;        /* slave+func+addr+count+bytecount+data+crc */
    if (total > max_len) return -1;

    out[0] = slave;
    out[1] = 0x0F;
    out[2] = (uint8_t)(addr >> 8);
    out[3] = (uint8_t)(addr & 0xFF);
    out[4] = (uint8_t)(count >> 8);
    out[5] = (uint8_t)(count & 0xFF);
    out[6] = (uint8_t)byte_count;

    /* pack bits */;
    memset(out + 7, 0, byte_count);
    for (int i = 0; i < count; i++)
        if (data[i]) out[7 + i / 8] |= (1 << (i % 8));

    uint16_t crc = modbus_crc16(out, total - 2);
    out[total - 2] = (uint8_t)(crc & 0xFF);
    out[total - 1] = (uint8_t)(crc >> 8);
    return total;
}

/* ─── Parse 0x01/0x02 bit response ──────────────────────────────────────────
── */
bool modbus_parse_bit_response(const uint8_t *data, int len,
                               uint8_t *bits_out, int max_bits)
{
    if (len < 5) return false;                    /* slave+func+bytecount+crc(2) */
    if (data[1] != 0x01 && data[1] != 0x02) return false;
    int byte_count = data[2];
    if (byte_count <= 0 || len < 5 + byte_count) return false;
    if (modbus_crc16(data, len) != 0) return false;

    for (int i = 0; i < max_bits; i++)
        bits_out[i] = (data[3 + i / 8] >> (i % 8)) & 0x01;
    return true;
}

bool modbus_parse_read_response(const uint8_t *data, int len, uint16_t *val, bool non_standard)
{
    int exp_len = non_standard ? 8 : 7;  /* non-std echoes 1 byte of address */

    if (len < exp_len) return false;
    if (data[1] != 0x03) return false;

    if (non_standard) {
        /* byte_count = 3 (echo: 1 byte addr + 2 bytes value) */
        if (data[2] < 3) return false;
        if (modbus_crc16(data, len) != 0) return false;
        *val = ((uint16_t)data[4] << 8) | data[5];
        return true;
    }

    /* ─── standard path (unchanged) ──────────────────────────────────────── */
    if (data[2] < 2) return false;

    if (modbus_crc16(data, len) == 0) goto accept;

    /* try swap CRC for non-standard CRC devices that echo standard registers */
    {
        uint8_t tmp_buf[256];
        if (len <= 256) {
```

```
            memcpy(tmp_buf, data, len);
            uint8_t t = tmp_buf[len - 2];
            tmp_buf[len - 2] = tmp_buf[len - 1];
            tmp_buf[len - 1] = t;
            if (modbus_crc16(tmp_buf, len) == 0) goto accept;
        }
    }
    return false;

accept:
    *val = ((uint16_t)data[3] << 8) | data[4];
    return true;
}

void modbus_frame_to_hex(const uint8_t *frame, int len, char *out, int max)
{
    int pos = 0;
    for (int i = 0; i < len && pos < max - 4; i++) {
        pos += snprintf(out + pos, max - pos, "%02X ", frame[i]);
    }
    out[pos > 0 ? pos - 1 : 0] = '\0';  /* trim trailing space */
}

/*
 * ethernet_comm.c — TCP/IP implementation
 *
 *  _WIN32        → WinSock2  (Windows simulator)
 *  __RTTHREAD__  → LWIP + RTL8201F RMII  (D133CBS hardware)
 */
#include "ethernet_comm.h"
#include <stdio.h>
#include <string.h>

/* ================================================================
 *  Platform: Windows (Simulator) — WinSock2
 * ================================================================ */
#ifdef _WIN32

#include <winsock2.h>
#include <ws2tcpip.h>
#pragma comment(lib, "ws2_32.lib")

static SOCKET g_sock        = INVALID_SOCKET;
static SOCKET g_listen_sock = INVALID_SOCKET;
static bool   g_connected   = false;
static bool   g_server_mode = false;
static bool   g_connecting  = false;
static bool   g_inited      = false;

bool ethernet_init(void)
{
    WSADATA wsa;
    if (WSAStartup(MAKEWORD(2, 2), &wsa) != 0) {
        fprintf(stderr, "[ETH] WSAStartup failed\n");
        return false;
    }
    g_inited = true;
    printf("[ETH] WinSock2 initialized\n");
    return true;
}

bool ethernet_connect(const char *ip, uint16_t port)
{
```

```
    struct sockaddr_in addr;

    if (!g_inited) {
        fprintf(stderr, "[ETH] Not initialized, call ethernet_init() first\n");
        return false;
    }

    /* disconnect previous connection if any */
    ethernet_disconnect();

    g_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (g_sock == INVALID_SOCKET) {
        fprintf(stderr, "[ETH] socket() failed\n");
        return false;
    }

    /* NON-BLOCKING: set before connect, returns immediately */
    u_long mode = 1;
    ioctlsocket(g_sock, FIONBIO, &mode);

    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = inet_addr(ip);
    addr.sin_port        = htons(port);

    int ret = connect(g_sock, (struct sockaddr *)&addr, sizeof(addr));
    if (ret == SOCKET_ERROR) {
        int err = WSAGetLastError();
        if (err != WSAEWOULDBLOCK) {
            fprintf(stderr, "[ETH] connect() to %s:%d failed, err=%d\n", ip, port, err);
            closesocket(g_sock);
            g_sock = INVALID_SOCKET;
            return false;
        }
        /* WSAEWOULDBLOCK = EINPROGRESS equivalent */
    }

    g_connecting = true;
    g_server_mode = false;
    printf("[ETH] Connecting to %s:%d (non-blocking)...\n", ip, port);
    return true;
}

bool ethernet_start_server(const char *local_ip, uint16_t port)
{
    struct sockaddr_in addr;
    int opt = 1;

    if (!g_inited) {
        fprintf(stderr, "[ETH] Not initialized\n");
        return false;
    }

    ethernet_disconnect();

    g_listen_sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (g_listen_sock == INVALID_SOCKET) {
        fprintf(stderr, "[ETH] socket() for server failed\n");
        return false;
    }

    setsockopt(g_listen_sock, SOL_SOCKET, SO_REUSEADDR,
               (const char *)&opt, sizeof(opt));
```

```
    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = inet_addr(local_ip);
    addr.sin_port        = htons(port);

    if (bind(g_listen_sock, (struct sockaddr *)&addr, sizeof(addr)) == SOCKET_ERROR) {
        fprintf(stderr, "[ETH] bind() to %s:%d failed\n", local_ip, port);
        closesocket(g_listen_sock);
        g_listen_sock = INVALID_SOCKET;
        return false;
    }

    if (listen(g_listen_sock, 1) == SOCKET_ERROR) {
        fprintf(stderr, "[ETH] listen() failed\n");
        closesocket(g_listen_sock);
        g_listen_sock = INVALID_SOCKET;
        return false;
    }

    /* non-blocking accept */
    u_long mode = 1;
    ioctlsocket(g_listen_sock, FIONBIO, &mode);

    g_server_mode = true;
    printf("[ETH] TCP Server listening on %s:%d\n", local_ip, port);
    return true;
}

bool ethernet_connect_poll(void)
{
    if (!g_connecting || g_sock == INVALID_SOCKET)
        return g_connected;

    fd_set wfds;
    struct timeval tv = {0, 0};

    FD_ZERO(&wfds);
    FD_SET(g_sock, &wfds);

    if (select(0, NULL, &wfds, NULL, &tv) <= 0)
        return false;

    g_connecting = false;
    g_connected = true;
    printf("[ETH] Connected!\n");
    return true;
}

bool ethernet_accept_poll(void)
{
    struct sockaddr_in client_addr;
    int addr_len = sizeof(client_addr);

    if (!g_server_mode || g_listen_sock == INVALID_SOCKET)
        return false;

    if (g_connected)
        return true;  /* already got a client */

    SOCKET client = accept(g_listen_sock,
                           (struct sockaddr *)&client_addr, &addr_len);
    if (client == INVALID_SOCKET) {
```

```
        int err = WSAGetLastError();
        if (err == WSAEWOULDBLOCK)
            return false;  /* no pending connection — normal */
        return false;
    }

    /* we have a client — stop listening, switch to connected mode */
    closesocket(g_listen_sock);
    g_listen_sock = INVALID_SOCKET;
    g_server_mode = false;
    g_sock = client;
    g_connected = true;

    /* client socket non-blocking */
    u_long mode2 = 1;
    ioctlsocket(g_sock, FIONBIO, &mode2);

    printf("[ETH] Server accepted client from %s:%d\n",
           inet_ntoa(client_addr.sin_addr),
           ntohs(client_addr.sin_port));
    return true;
}

void ethernet_disconnect(void)
{
    if (g_sock != INVALID_SOCKET) {
        closesocket(g_sock);
        g_sock = INVALID_SOCKET;
    }
    if (g_listen_sock != INVALID_SOCKET) {
        closesocket(g_listen_sock);
        g_listen_sock = INVALID_SOCKET;
    }
    g_connected = false;
    g_server_mode = false;
    g_connecting = false;
    printf("[ETH] Disconnected\n");
}

int ethernet_send(const uint8_t *data, uint16_t len)
{
    if (g_sock == INVALID_SOCKET || !g_connected) return -1;
    int sent = send(g_sock, (const char *)data, (int)len, 0);
    if (sent == SOCKET_ERROR) {
        int err = WSAGetLastError();
        if (err == WSAEWOULDBLOCK)
            return 0;
        if (err == WSAECONNRESET || err == WSAECONNABORTED) {
            ethernet_disconnect();
        }
        return -1;
    }
    return sent;
}

int ethernet_recv(uint8_t *buf, uint16_t buf_size)
{
    if (g_sock == INVALID_SOCKET || !g_connected) return -1;

    int n = recv(g_sock, (char *)buf, (int)buf_size, 0);
    if (n > 0) return n;
    if (n == 0) {
        /* graceful close by peer */
```

```
        ethernet_disconnect();
        return -1;
    }
    int err = WSAGetLastError();
    if (err == WSAEWOULDBLOCK)
        return 0;            /* no data available — normal */
    /* real error */
    ethernet_disconnect();
    return -1;
}

bool ethernet_is_connected(void)
{
    return g_connected;
}

void ethernet_cleanup(void)
{
    ethernet_disconnect();
    if (g_inited) {
        WSACleanup();
        g_inited = false;
    }
    printf("[ETH] Cleaned up\n");
}

/* ================================================================
 *  Platform: D133CBS (Hardware) — RT-Thread + LWIP + RTL8201F
 *
 *  BSP already inits MAC (aicmac_init), PHY (aicphy_init), LWIP netif,
 *  and DHCP before main(). We just use BSD-socket API directly.
 *
 *  Reference: bsp/examples/test-lwip/test-tcp/test_tcp_socket.c
 * ================================================================ */
#elif defined(__RTTHREAD__) || defined(RT_USING_LWIP)

#include <sys/socket.h>
#include <netdb.h>
#include <fcntl.h>
#include <errno.h>
#include <aic_osal.h>
#include "lwip/netif.h"
#include "lwip/ip_addr.h"

/* ---- ring buffer: TCP thread → LVGL timer ---- */
#define RX_RING_SIZE    4096
static uint8_t g_rx_ring[RX_RING_SIZE];
static volatile int g_rx_wr = 0;
static volatile int g_rx_rd = 0;

static void ring_write(const uint8_t *data, int len) {
    for (int i = 0; i < len; i++) {
        g_rx_ring[g_rx_wr] = data[i];
        g_rx_wr = (g_rx_wr + 1) & (RX_RING_SIZE - 1);
    }
}
int ethernet_ring_read(uint8_t *buf, int max_len) {
    int avail = (g_rx_wr - g_rx_rd) & (RX_RING_SIZE - 1);
    if (avail > max_len) avail = max_len;
    for (int i = 0; i < avail; i++) {
        buf[i] = g_rx_ring[g_rx_rd];
        g_rx_rd = (g_rx_rd + 1) & (RX_RING_SIZE - 1);
    }
```

```
    return avail;
}

/* ---- shared between LVGL thread and TCP worker ---- */
static volatile int  g_sock        = -1;
static volatile int  g_listen_sock = -1;
static volatile bool g_connected   = false;
static volatile bool g_server_mode = false;
static volatile bool g_run         = false;

/* ---- TCP worker thread ---- */
static void tcp_worker(void *arg)
{
    uint8_t buf[512];
    struct sockaddr_in addr;
    socklen_t addr_len;
    int sock, lsn, n;

    (void)arg;

    while (g_run) {
        lsn  = g_listen_sock;
        sock = g_sock;

        /* Server: accept */
        if (g_server_mode && lsn >= 0 && !g_connected) {
            addr_len = sizeof(addr);
            n = accept(lsn, (struct sockaddr *)&addr, &addr_len);
            if (n >= 0) {
                closesocket(lsn);
                g_listen_sock = -1;
                g_server_mode = false;
                g_sock = n;
                g_connected = true;
                printf("[ETH] Accepted %s:%d\n",
                       inet_ntoa(addr.sin_addr), ntohs(addr.sin_port));
            } else {
                aicos_msleep(200);
            }
            continue;
        }

        /* Connected: recv */
        if (g_connected && sock >= 0) {
            n = recv(sock, buf, sizeof(buf), 0);
            if (n > 0) {
                ring_write(buf, n);
            } else {
                printf("[ETH] recv closed (%d)\n", n);
                closesocket(sock);
                g_sock = -1;
                g_connected = false;
            }
            continue;
        }

        /* Idle */
        aicos_msleep(200);
    }
}

bool ethernet_init(void)
{
```

```
    struct netif *netif = netif_list;
    ip4_addr_t ipaddr, netmask, gw;

    if (!netif) {
        printf("[ETH] No netif found\n");
        return false;
    }

    ip4addr_aton(AIC_DEV_GMAC0_IPADDR,  &ipaddr);
    ip4addr_aton(AIC_DEV_GMAC0_NETMASK, &netmask);
    ip4addr_aton(AIC_DEV_GMAC0_GW,      &gw);
    netif_set_ipaddr(netif, &ipaddr);
    netif_set_netmask(netif, &netmask);
    netif_set_gw(netif, &gw);
    printf("[ETH] IP: %s/%s gw %s\n",
           AIC_DEV_GMAC0_IPADDR, AIC_DEV_GMAC0_NETMASK, AIC_DEV_GMAC0_GW);

    g_run = true;
    aicos_thread_create("tcp_work", 4096, 22, tcp_worker, NULL);
    printf("[ETH] TCP worker started\n");
    return true;
}

bool ethernet_connect(const char *ip, uint16_t port)
{
    struct sockaddr_in addr;
    int sock;

    ethernet_disconnect();

    sock = socket(AF_INET, SOCK_STREAM, 0);
    if (sock < 0) { printf("[ETH] socket failed\n"); return false; }

    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_len         = sizeof(struct sockaddr_in);
    addr.sin_addr.s_addr = inet_addr(ip);
    addr.sin_port        = htons(port);

    printf("[ETH] Connecting to %s:%d ...\n", ip, port);
    if (connect(sock, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        printf("[ETH] connect failed, errno=%d\n", errno);
        closesocket(sock);
        return false;
    }

    g_sock = sock;
    g_connected = true;
    printf("[ETH] Connected!\n");
    return true;
}

bool ethernet_start_server(const char *local_ip, uint16_t port)
{
    struct sockaddr_in addr;
    int opt = 1, lsn;

    ethernet_disconnect();

    lsn = socket(AF_INET, SOCK_STREAM, 0);
    if (lsn < 0) { printf("[ETH] socket failed\n"); return false; }

    setsockopt(lsn, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
```

```
    memset(&addr, 0, sizeof(addr));
    addr.sin_family      = AF_INET;
    addr.sin_len         = sizeof(struct sockaddr_in);
    addr.sin_addr.s_addr = inet_addr(local_ip);
    addr.sin_port        = htons(port);

    if (bind(lsn, (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        printf("[ETH] bind failed, errno=%d\n", errno);
        closesocket(lsn);
        return false;
    }
    if (listen(lsn, 1) < 0) {
        printf("[ETH] listen failed\n");
        closesocket(lsn);
        return false;
    }

    g_listen_sock = lsn;
    g_server_mode = true;
    printf("[ETH] Server listening on %s:%d\n", local_ip, port);
    return true;
}

bool ethernet_connect_poll(void) { return g_connected; }
bool ethernet_accept_poll(void)  { return g_connected; }

void ethernet_disconnect(void)
{
    int s = g_sock, l = g_listen_sock;
    g_sock = -1;
    g_listen_sock = -1;
    g_connected = false;
    g_server_mode = false;
    if (s >= 0) closesocket(s);
    if (l >= 0) closesocket(l);
    printf("[ETH] Disconnected\n");
}

int ethernet_send(const uint8_t *data, uint16_t len)
{
    int s = g_sock;
    if (s < 0 || !g_connected) return -1;
    return send(s, data, (int)len, 0);
}

int ethernet_recv(uint8_t *buf, uint16_t buf_size)
{
    return ethernet_ring_read(buf, buf_size);
}

bool ethernet_is_connected(void) { return g_connected; }

void ethernet_cleanup(void)
{
    g_run = false;
    aicos_msleep(300);
    ethernet_disconnect();
    printf("[ETH] Cleaned up\n");
}

#else
    /* POSIX / unknown — stub */
```

```
    #warning "Unknown platform for ethernet_comm.c"
    bool ethernet_init(void) { return false; }
    bool ethernet_connect(const char *ip, uint16_t port) { (void)ip; (void)port; return false; }
    bool ethernet_connect_poll(void) { return false; }
    bool ethernet_start_server(const char *ip, uint16_t port) { (void)ip; (void)port; return false; }
    bool ethernet_accept_poll(void) { return false; }
    void ethernet_disconnect(void) {}
    int  ethernet_send(const uint8_t *data, uint16_t len) { (void)data; (void)len; return -1; }
    int  ethernet_recv(uint8_t *buf, uint16_t buf_size) { (void)buf; (void)buf_size; return -1; }
    bool ethernet_is_connected(void) { return false; }
    void ethernet_cleanup(void) {}
#endif

/*
 * Copyright (C) 2026 ArtInChip Technology Co., Ltd.
 *
 * OneNET HTTP OTA implementation.
 *
 * Device side:  uses webclient + OTA engine (real flash write).
 * Simulator side: stubs that simulate progress (UI can be previewed).
 */

#include "onenet_ota.h"
#include <string.h>
#include <stdio.h>
#include <stdlib.h>

/* rtconfig.h is force-included by the RT-Thread build system;
 * on the simulator it doesn't exist, so LPKG_* macros stay undefined
 * and ONENET_OTA_REAL correctly falls back to 0. */
#if !defined(LPKG_USING_WEBCLIENT) && !defined(LPKG_USING_HTTP_OTA) && !defined(_WIN32)
#include <rtconfig.h>
#endif

#ifdef _WIN32
#include <winsock2.h>
#include <windows.h>
#endif

/* ---- Platform detection ------------------------------------------------ */
#if defined(LPKG_USING_WEBCLIENT) && defined(LPKG_USING_HTTP_OTA)
   #define ONENET_OTA_REAL  1
#else
   #define ONENET_OTA_REAL  0
#endif

#if ONENET_OTA_REAL
   #include <rtthread.h>
   #include "webclient.h"
   #include "ota.h"
   #include "absystem_os.h"
#endif

/* ---- State ------------------------------------------------------------- */
static onenet_ota_state_t g_state       = ONENET_OTA_IDLE;
static char               g_new_version[32];
static int                g_new_size    = 0;
static char               g_download_url[256];
static int                g_task_id     = 0;
static char               g_cur_version[32] = "v1.0.0";
```

```c
static onenet_ota_progress_cb g_progress_cb = NULL;

/* ─── Macros ──────────────
```

```
/* ─── Send HTTP request over EXISTING socket. post_body!=NULL → POST */
static int _http_req_sock(int sock, const char *http_host,
                          const char *method, const char *path,
                          const char *extra_headers,
                          const char *post_body,
                          char *resp_buf, int resp_buf_len)
{
    char req[1024];
    int req_len;
    if (extra_headers && extra_headers[0]) {
        req_len = snprintf(req, sizeof(req),
            "%s %s HTTP/1.1\r\n"
            "Host: %s\r\n"
            "%s"
            "Connection: keep-alive\r\n"
            "\r\n",
            method, path, http_host, extra_headers);
    } else {
        req_len = snprintf(req, sizeof(req),
            "%s %s HTTP/1.1\r\n"
            "Host: %s\r\n"
            "Connection: keep-alive\r\n"
            "\r\n",
            method, path, http_host);
    }

    if (lwip_send(sock, req, req_len, 0) != req_len) return -1;

    /* send POST body after headers */
    if (post_body) {
        int body_len = strlen(post_body);
        if (lwip_send(sock, post_body, body_len, 0) != body_len) return -1;
    }

    memset(resp_buf, 0, resp_buf_len);
    int total = 0;
    char *hdr_end = NULL;

    while (total < resp_buf_len - 1) {
        int n = lwip_recv(sock, resp_buf + total, resp_buf_len - 1 - total, 0);
        if (n <= 0) return -1;
        total += n;
        resp_buf[total] = '\0';
        hdr_end = strstr(resp_buf, "\r\n\r\n");
        if (hdr_end) break;
    }
    if (!hdr_end) return -1;

    /* parse Content-Length */
    int content_len = -1;
    char *cl = strstr(resp_buf, "Content-Length:");
    if (!cl) cl = strstr(resp_buf, "content-length:");
    if (cl) { cl += 15; while (*cl == ' ') cl++; content_len = atoi(cl); }

    int body_start = (int)(hdr_end + 4 - resp_buf);
    int body_so_far = total - body_start;

    if (content_len > 0) {
        while (body_so_far < content_len && total < resp_buf_len - 1) {
            int n = lwip_recv(sock, resp_buf + total, resp_buf_len - 1 - total, 0);
            if (n <= 0) break;
            total += n;
```

```
            body_so_far = total - body_start;
        }
    } else {
        while (total < resp_buf_len - 1) {
            int n = lwip_recv(sock, resp_buf + total, resp_buf_len - 1 - total, 0);
            if (n <= 0) break;
            total += n;
        }
    }

    if (body_so_far <= 0) return -1;
    memmove(resp_buf, resp_buf + body_start, body_so_far);
    resp_buf[body_so_far] = '\0';
    return body_so_far;
}

/* ---- Minimal JSON parse ----------------------------------------------- */
static int json_get_str(const char *json, const char *key, char *out, int out_len)
{
    char search[64];
    snprintf(search, sizeof(search), "\"%s\":\"", key);
    const char *p = strstr(json, search);
    if (!p) return -1;
    p += strlen(search);
    int i = 0;
    while (*p && *p != '"' && i < out_len - 1) out[i++] = *p++;
    out[i] = '\0';
    return i;
}

static int json_get_int(const char *json, const char *key, int *val)
{
    char search[64];
    snprintf(search, sizeof(search), "\"%s\":", key);
    const char *p = strstr(json, search);
    if (!p) return -1;
    p += strlen(search);
    *val = atoi(p);
    return 0;
}

#define ONENET_IP     "183.230.40.33"   /* TCP connect target */
#define ONENET_DOMAIN "iot-api.heclouds.com"  /* HTTP Host header */
#define ONENET_PORT   80

/* forward */
static int _http_request_once(const char *conn_host, const char *http_host,
                              int port, const char *method, const char *path,
                              const char *extra_hdr, const char *post_body,
                              char *resp, int resp_sz);

/* ---- Version check ---------------------------------------------------- */
int onenet_ota_check(void)
{
    g_state = ONENET_OTA_CHECKING;

    char path[256];
    snprintf(path, sizeof(path),
        "/fuse-ota/%s/%s/check?type=1&version=1.0",
        ONENET_PROD_ID, ONENET_DEVICE_ID);
```

```
    char extra_hdr[256];
    snprintf(extra_hdr, sizeof(extra_hdr),
        "Authorization: %s\r\n", ONENET_AUTH_HEADER);

    char body[1024];
    int body_len = _http_request_once(ONENET_IP, ONENET_DOMAIN, ONENET_PORT,
                                      "GET", path, extra_hdr, NULL,
                                      body, sizeof(body));
    if (body_len <= 0) {
        g_state = ONENET_OTA_ERROR;
        return -1;
    }

    /* parse JSON */
    int code = -1;
    json_get_int(body, "code", &code);
    if (code != 0) {
        g_state = ONENET_OTA_UP_TO_DATE;
        return 0;
    }

    if (json_get_str(body, "target", g_new_version, sizeof(g_new_version)) < 0) {
        g_state = ONENET_OTA_UP_TO_DATE;
        return 0;
    }

    int tid = 0;
    json_get_int(body, "tid", &tid);
    json_get_int(body, "size", &g_new_size);
    g_task_id = tid;

    /* build download URL path */
    snprintf(g_download_url, sizeof(g_download_url),
        "/fuse-ota/%s/%s/%d/download",
        ONENET_PROD_ID, ONENET_DEVICE_ID, tid);

    /* compare versions */
    if (strcmp(g_new_version, g_cur_version) == 0) {
        g_state = ONENET_OTA_UP_TO_DATE;
        return 0;
    }

    g_state = ONENET_OTA_AVAILABLE;
    return 1;
}

/* ─── Download one Range chunk over persistent socket ──────────────────── */
static int _download_range_sock(int sock, const char *path,
                                int start, int end,
                                char *data_buf, int buf_size)
{
    static char raw[RECV_BUF_LEN];

    char extra_hdr[384];
    snprintf(extra_hdr, sizeof(extra_hdr),
        "Authorization: %s\r\n"
        "Range: bytes=%d-%d\r\n",
        ONENET_AUTH_HEADER, start, end);

    int raw_len = _http_req_sock(sock, ONENET_DOMAIN, "GET",
                                 path, extra_hdr, NULL,
                                 raw, sizeof(raw));
    printf("[OTA] range %d-%d, raw_len=%d\n", start, end, raw_len);
```

```
    if (raw_len <= 0) return -1;

    int copy_len = (raw_len < buf_size) ? raw_len : buf_size;
    if (copy_len <= 0) return -1;
    memcpy(data_buf, raw, copy_len);
    return copy_len;
}

/* ─── One-shot HTTP request (for check / report status) ──────────────────── */
static int _http_request_once(const char *conn_host, const char *http_host,
                              int port, const char *method, const char *path,
                              const char *extra_hdr, const char *post_body,
                              char *resp, int resp_sz)
{
    int sock = _tcp_connect(conn_host, port);
    if (sock < 0) return -1;
    int len = _http_req_sock(sock, http_host, method, path, extra_hdr, post_body, resp, resp_sz);
    lwip_close(sock);
    return len;
}

/* ─── Report status to OneNET ──────────────────────────────────────────────────────────────────────── */
static void _report_status(int step)
{
    char path[256];
    snprintf(path, sizeof(path),
        "/fuse-ota/%s/%s/%d/status",
        ONENET_PROD_ID, ONENET_DEVICE_ID, g_task_id);

    char post_data[64];
    snprintf(post_data, sizeof(post_data), "{\"step\":%d}", step);

    char extra_hdr[384];
    snprintf(extra_hdr, sizeof(extra_hdr),
        "Authorization: %s\r\n"
        "Content-Type: application/json\r\n"
        "Content-Length: %d\r\n",
        ONENET_AUTH_HEADER, (int)strlen(post_data));

    char resp[512];
    int r = _http_request_once(ONENET_IP, ONENET_DOMAIN, ONENET_PORT,
                               "POST", path, extra_hdr, post_data,
                               resp, sizeof(resp));
    printf("[OTA] report step=%d, ret=%d, resp=%.200s\n", step, r, resp);
}

/* ─── Firmware download ──────────────────────────────────────────────────────────────────────────────── */
int onenet_ota_download(onenet_ota_progress_cb progress_cb)
{
    if (g_state != ONENET_OTA_AVAILABLE) return -1;
    g_progress_cb = progress_cb;
    g_state = ONENET_OTA_DOWNLOADING;

    if (progress_cb) progress_cb(0, "正在准备下载...");

    /* open ONE persistent TCP connection for all Range requests */
    printf("[OTA] opening TCP to %s\n", ONENET_IP);
    int sock = _tcp_connect(ONENET_IP, ONENET_PORT);
    if (sock < 0) {
        printf("[OTA] TCP connect failed\n");
        if (progress_cb) progress_cb(0, "连接服务器失败");
```

```
        g_state = ONENET_OTA_ERROR;
        return -1;
    }

    /* init OTA engine */
    if (progress_cb) progress_cb(1, "正在初始化 OTA 引擎...");
    int ret = ota_init();
    if (ret != 0) {
        printf("[OTA] ota_init failed ret=%d\n", ret);
        if (progress_cb) progress_cb(0, "OTA 初始化失败");
        g_state = ONENET_OTA_ERROR;
        lwip_close(sock);
        return -1;
    }

    static char chunk[OTA_BUFF_LEN];
    int downloaded = 0, last_pct = -1;

    printf("[OTA] dl start: path=%s, total_size=%d, chunk=%d\n",
           g_download_url, g_new_size, OTA_BUFF_LEN);

    while (downloaded < g_new_size) {
        int end = downloaded + OTA_BUFF_LEN - 1;
        if (end >= g_new_size) end = g_new_size - 1;

        int len = _download_range_sock(sock, g_download_url,
                                       downloaded, end,
                                       chunk, OTA_BUFF_LEN);
        if (len <= 0) {
            printf("[OTA] download failed at offset %d, len=%d\n", downloaded, len);
            if (progress_cb) progress_cb(0, "固件下载失败, 请检查网络");
            g_state = ONENET_OTA_ERROR;
            lwip_close(sock);
            ota_deinit();
            return -1;
        }

        ret = ota_shard_download_fun(chunk, len);
        if (ret < 0) {
            if (progress_cb) progress_cb(0, "固件写入失败");
            g_state = ONENET_OTA_ERROR;
            lwip_close(sock);
            ota_deinit();
            return -1;
        }

        downloaded += len;

        int pct = downloaded * 100 / g_new_size;
        if (pct != last_pct && progress_cb) {
            last_pct = pct;
            char status[64];
            snprintf(status, sizeof(status), "正在下载固件... %d%%", pct);
            progress_cb(pct, status);
        }
    }
    lwip_close(sock);

    /* 3. verify — show text first, THEN block LVGL from SPI NOR. */
    if (progress_cb)
        progress_cb(99,
"\xe6\xad\xa3\xe5\x9c\xa8\xe6\xa3\x80\xe6\x9f\xa5\xe5\x9b\xba\xe4\xbb\xb6...");
    rt_thread_delay(rt_tick_from_millisecond(150));
```

```
    g_state = ONENET_OTA_VERIFYING;
    rt_thread_delay(rt_tick_from_millisecond(50));

    /* 4. finalize A/B switch */
    ret = aic_upgrade_end();
    if (ret != 0) {
        g_state = ONENET_OTA_ERROR;
        ota_deinit();
        return -1;
    }

    /* 5. report success to OneNET */
    _report_status(101);
    _report_status(201);

    ota_deinit();

    /* 6. hide UI, print success, reboot */
    g_state = ONENET_OTA_SUCCESS;
    rt_kprintf("[OTA] Upgrade OK, rebooting...\n");
    rt_thread_delay(rt_tick_from_millisecond(2000));
    rt_hw_cpu_reset();

    return 0;
}

int onenet_ota_init(void)
{
    /* read current version from system params if available */
#if defined(APP_VERION)
    strncpy(g_cur_version, APP_VERION, sizeof(g_cur_version) - 1);
#endif
    g_state = ONENET_OTA_IDLE;
    return 0;
}

/* ================================================================================
================================
 *  WINDOWS SIMULATOR  (real HTTP to OneNET, no flash write)
 *
 *  Uses WinSock to send real GET /fuse-ota/.../check to OneNET.
 *  Download is still simulated (PC can't write SPI NOR).
 * ================================================================================
================================ */
#else  /* !ONENET_OTA_REAL */

#ifdef _WIN32

static char g_http_buf[4096];

static int _http_get(const char *host, int port, const char *path,
                     const char *auth, char *resp, int resp_sz)
{
    WSADATA wsa;
    SOCKET sock = INVALID_SOCKET;
    struct hostent *he;
    struct sockaddr_in addr;
    int ret = -1;

    if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) return -1;

    he = gethostbyname(host);
    if (!he) goto out;
```

```
    sock = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (sock == INVALID_SOCKET) goto out;

    addr.sin_family = AF_INET;
    addr.sin_port   = htons((u_short)port);
    addr.sin_addr   = *(struct in_addr *)he->h_addr;

    /* 5 second timeout */
    int to = 5000;
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, (const char *)&to, sizeof(to));
    setsockopt(sock, SOL_SOCKET, SO_SNDTIMEO, (const char *)&to, sizeof(to));

    if (connect(sock, (struct sockaddr *)&addr, sizeof(addr)) != 0) goto out;

    /* build HTTP request */
    char req[1024];
    int req_len;
    if (auth && auth[0]) {
        req_len = snprintf(req, sizeof(req),
            "GET %s HTTP/1.1\r\n"
            "Host: %s\r\n"
            "Authorization: %s\r\n"
            "User-Agent: A1-OTA-Simulator/1.0\r\n"
            "Connection: close\r\n\r\n",
            path, host, auth);
    } else {
        req_len = snprintf(req, sizeof(req),
            "GET %s HTTP/1.1\r\n"
            "Host: %s\r\n"
            "User-Agent: A1-OTA-Simulator/1.0\r\n"
            "Connection: close\r\n\r\n",
            path, host);
    }

    if (send(sock, req, req_len, 0) != req_len) goto out;

    /* receive response */
    memset(resp, 0, resp_sz);
    int total = 0;
    while (total < resp_sz - 1) {
        int n = recv(sock, resp + total, resp_sz - 1 - total, 0);
        if (n <= 0) break;
        total += n;
    }

    /* skip HTTP header → find body after \r\n\r\n */
    char *body = strstr(resp, "\r\n\r\n");
    if (!body) { ret = total; goto out; }

    body += 4;
    int body_len = total - (int)(body - resp);
    memmove(resp, body, body_len);
    resp[body_len] = '\0';
    ret = body_len;

out:
    if (sock != INVALID_SOCKET) closesocket(sock);
    WSACleanup();
    return ret;
}

/* ─── Simple JSON number parser (avoids cJSON dependency) ──────────── */
```

```c
static int _jint(const char *j, const char *key, int *val)
{
    char s[64];
    snprintf(s, sizeof(s), "\"%s\":", key);
    const char *p = strstr(j, s);
    if (!p) return -1;
    p += strlen(s);
    *val = atoi(p);
    return 0;
}

static int _jstr(const char *j, const char *key, char *out, int n)
{
    char s[64];
    snprintf(s, sizeof(s), "\"%s\":\"", key);
    const char *p = strstr(j, s);
    if (!p) return -1;
    p += strlen(s);
    int i = 0;
    while (*p && *p != '"' && i < n - 1) out[i++] = *p++;
    out[i] = '\0';
    return i;
}

#endif /* _WIN32 */

int onenet_ota_init(void)
{
    printf("[OTA-sim] init OK (real HTTP to OneNET)\n");
    g_state = ONENET_OTA_IDLE;
    return 0;
}

int onenet_ota_check(void)
{
#ifdef _WIN32
    g_state = ONENET_OTA_CHECKING;

    char path[256];
    snprintf(path, sizeof(path),
        "/fuse-ota/%s/%s/check?type=1&version=1.0",
        ONENET_PROD_ID, ONENET_DEVICE_ID);

    printf("[OTA-sim] GET http://iot-api.heclouds.com%s\n", path);

    int len = _http_get("iot-api.heclouds.com", 80, path,
                        ONENET_AUTH_HEADER, g_http_buf, sizeof(g_http_buf));
    if (len <= 0) {
        printf("[OTA-sim] HTTP request failed (len=%d)\n", len);
        g_state = ONENET_OTA_ERROR;
        return -1;
    }

    printf("[OTA-sim] response: %s\n", g_http_buf);

    /* parse JSON */
    int code = -1;
    _jint(g_http_buf, "code", &code);
    if (code != 0) {
        printf("[OTA-sim] server code=%d (no update available)\n", code);
        g_state = ONENET_OTA_UP_TO_DATE;
        return 0;
    }
```

```
    /* extract data fields from nested "data":{...} */
    const char *data_start = strstr(g_http_buf, "\"data\":{");
    if (!data_start) {
        printf("[OTA-sim] no data field in response\n");
        g_state = ONENET_OTA_UP_TO_DATE;
        return 0;
    }
    /* find the closing } for the data object */
    const char *p = data_start;
    int depth = 0;
    while (*p) {
        if (*p == '{') depth++;
        else if (*p == '}') { depth--; if (depth == 0) break; }
        p++;
    }
    /* copy just the data object */
    int data_len = (int)(p - data_start) + 1;
    if (data_len >= 2048) data_len = 2047;
    char data_json[2048];
    memcpy(data_json, data_start, data_len);
    data_json[data_len] = '\0';

    if (_jstr(data_json, "target", g_new_version, sizeof(g_new_version)) < 0) {
        printf("[OTA-sim] target version not found in data\n");
        g_state = ONENET_OTA_UP_TO_DATE;
        return 0;
    }

    int tid = 0, sz = 0;
    _jint(data_json, "tid", &tid);
    _jint(data_json, "size", &sz);
    g_new_size = sz;
    g_task_id  = tid;

    /* build download URL path */
    snprintf(g_download_url, sizeof(g_download_url),
        "/fuse-ota/%s/%s/%d/download",
        ONENET_PROD_ID, ONENET_DEVICE_ID, tid);

    printf("[OTA-sim] new version: %s, size: %d bytes\n",
           g_new_version, g_new_size);

    /* compare versions */
    if (strcmp(g_new_version, g_cur_version) == 0) {
        g_state = ONENET_OTA_UP_TO_DATE;
        return 0;
    }

    g_state = ONENET_OTA_AVAILABLE;
    return 1;
#else
    /* Non-Windows stub: fake data */
    g_state = ONENET_OTA_CHECKING;
    strncpy(g_new_version, "v2.0.0", sizeof(g_new_version) - 1);
    g_new_size = 345678;
    g_task_id  = 99999;
    if (strcmp(g_new_version, g_cur_version) == 0) {
        g_state = ONENET_OTA_UP_TO_DATE;
        return 0;
    }
    g_state = ONENET_OTA_AVAILABLE;
    return 1;
```

```
#endif
}

int onenet_ota_download(onenet_ota_progress_cb cb)
{
    printf("[OTA-sim] downloading %s (%d bytes) — simulated\n",
           g_new_version, g_new_size);
    g_state = ONENET_OTA_DOWNLOADING;

    /* simulate download progress over ~6 seconds */
    for (int i = 0; i <= 100; i += 5) {
        if (cb) {
            char status[64];
            snprintf(status, sizeof(status), "模拟下载中... %d%%", i);
            cb(i, status);
        }
#ifdef _WIN32
        Sleep(300);
#endif
    }

    if (cb) cb(95, "\xe6\xad\xa3\xe5\x9c\xa8\xe6\xa3\x80\xe6\x9f\xa5\xe5\x9b\xba\xe4\xbb\xb6...");
    g_state = ONENET_OTA_VERIFYING;
#ifdef _WIN32
    Sleep(1000);
#endif

    if (cb) cb(100, "升级成功，即将重启...");
    g_state = ONENET_OTA_SUCCESS;
#ifdef _WIN32
    Sleep(2000);
#endif

    return 0;
}

#endif /* ONENET_OTA_REAL */

/* ============================================================================
 * COMMON HELPERS (both real + stub)
 * ============================================================================ */

const char *onenet_ota_get_new_version(void)
{
    return g_new_version;
}

int onenet_ota_get_new_size(void)
{
    return g_new_size;
}

const char *onenet_ota_get_current_version(void)
{
    return g_cur_version;
}

onenet_ota_state_t onenet_ota_get_state(void)
{
    return g_state;
}
```

```c
/*
 * Copyright (C) 2026 ArtInChip Technology Co., Ltd.
 *
 * OTA UI — LVGL dynamic overlay (popup, progress bar, error).
 * Uses aic_get_multi_lang_font() for Chinese-capable fonts.
 * All widgets on lv_layer_top() — never touches UIBuilder screens.
 */

#include "ota_ui.h"
#include "lvgl.h"
#include "aic_lang.h"
#include "font_list.h"
#include <string.h>
#include <stdio.h>

static inline const lv_font_t *_F(int sz) {
    return aic_get_multi_lang_font(sz);
}

static lv_obj_t *g_overlay   = NULL;
static lv_obj_t *g_panel     = NULL;
static lv_obj_t *g_title     = NULL;
static lv_obj_t *g_info      = NULL;
static lv_obj_t *g_bar       = NULL;
static lv_obj_t *g_status    = NULL;
static lv_obj_t *g_btn_ok    = NULL;
static lv_obj_t *g_btn_cancel = NULL;

static ota_ui_btn_cb_t g_confirm_cb = NULL;  /* "立即升级" */
static ota_ui_btn_cb_t g_cancel_cb  = NULL;

static void _ensure_created(void);
static void _show_buttons(int ok, int cancel,
                          const char *ok_txt, const char *cancel_txt,
                          lv_event_cb_t ok_cb);
static void _hide_all(void);

/* ─── dismiss: just hide ──────────────────────────────────────────────── */
static void _on_dismiss(lv_event_t *e) {
    (void)e;
    ota_ui_hide();
}

/* ─── confirm: user clicked "立即升级" ──────────────────────────────── */
static void _on_confirm(lv_event_t *e) {
    (void)e;
    if (g_confirm_cb) g_confirm_cb();
}

/* ─── cancel: user clicked "稍后" ─────────────────────────────────────── */
static void _on_cancel(lv_event_t *e) {
    (void)e;
    ota_ui_hide();
    if (g_cancel_cb) g_cancel_cb();
}

#define PANEL_W  380
#define PANEL_H  270

static void _ensure_created(void)
```

```
{
    if (g_overlay) return;

    g_overlay = lv_obj_create(lv_layer_top());
    lv_obj_set_size(g_overlay, lv_pct(100), lv_pct(100));
    lv_obj_set_style_bg_color(g_overlay, lv_color_hex(0x000000), 0);
    lv_obj_set_style_bg_opa(g_overlay, LV_OPA_60, 0);
    lv_obj_set_style_border_width(g_overlay, 0, 0);
    lv_obj_set_style_radius(g_overlay, 0, 0);
    lv_obj_clear_flag(g_overlay, LV_OBJ_FLAG_SCROLLABLE);
    lv_obj_add_flag(g_overlay, LV_OBJ_FLAG_HIDDEN);

    g_panel = lv_obj_create(g_overlay);
    lv_obj_set_size(g_panel, PANEL_W, PANEL_H);
    lv_obj_center(g_panel);
    lv_obj_set_style_bg_color(g_panel, lv_color_hex(0xFFFFFF), 0);
    lv_obj_set_style_bg_opa(g_panel, LV_OPA_COVER, 0);
    lv_obj_set_style_border_width(g_panel, 0, 0);
    lv_obj_set_style_radius(g_panel, 12, 0);
    lv_obj_set_style_shadow_width(g_panel, 20, 0);
    lv_obj_set_style_shadow_opa(g_panel, LV_OPA_40, 0);
    lv_obj_clear_flag(g_panel, LV_OBJ_FLAG_SCROLLABLE);

    g_title = lv_label_create(g_panel);
    lv_obj_set_style_text_color(g_title, lv_color_hex(0x1A1A2E), 0);
    lv_obj_set_style_text_font(g_title, _F(16), 0);
    lv_label_set_text(g_title, "");
    lv_obj_align(g_title, LV_ALIGN_TOP_MID, 0, 16);

    g_info = lv_label_create(g_panel);
    lv_obj_set_style_text_color(g_info, lv_color_hex(0x555555), 0);
    lv_obj_set_style_text_font(g_info, _F(16), 0);
    lv_label_set_text(g_info, "");
    lv_obj_align(g_info, LV_ALIGN_TOP_MID, 0, 50);
    lv_obj_set_width(g_info, PANEL_W - 40);

    g_bar = lv_bar_create(g_panel);
    lv_obj_set_size(g_bar, PANEL_W - 60, 16);
    lv_obj_align(g_bar, LV_ALIGN_CENTER, 0, 10);
    lv_bar_set_range(g_bar, 0, 100);
    lv_bar_set_value(g_bar, 0, LV_ANIM_OFF);
    lv_obj_set_style_bg_color(g_bar, lv_color_hex(0xE8E8E8), 0);
    lv_obj_set_style_radius(g_bar, 8, 0);
    lv_obj_set_style_radius(g_bar, 8, LV_PART_INDICATOR);
    lv_obj_set_style_bg_color(g_bar, lv_color_hex(0x2196F3), LV_PART_INDICATOR);

    g_status = lv_label_create(g_panel);
    lv_obj_set_style_text_color(g_status, lv_color_hex(0x888888), 0);
    lv_obj_set_style_text_font(g_status, _F(16), 0);
    lv_label_set_text(g_status, "");
    lv_obj_align(g_status, LV_ALIGN_CENTER, 0, 35);
}

static void _show_buttons(int ok, int cancel,
                          const char *ok_txt, const char *cancel_txt,
                          lv_event_cb_t ok_cb)
{
    if (g_btn_ok)     { lv_obj_del(g_btn_ok);     g_btn_ok     = NULL; }
    if (g_btn_cancel) { lv_obj_del(g_btn_cancel); g_btn_cancel = NULL; }

    int bw = 120, bh = 40;

    if (ok) {
```

```
        g_btn_ok = lv_btn_create(g_panel);
        lv_obj_set_size(g_btn_ok, bw, bh);
        lv_obj_set_style_radius(g_btn_ok, 8, 0);
        lv_obj_set_style_bg_color(g_btn_ok, lv_color_hex(0x2196F3), 0);
        lv_obj_t *l = lv_label_create(g_btn_ok);
        lv_label_set_text(l, ok_txt ? ok_txt : "OK");
        lv_obj_set_style_text_color(l, lv_color_hex(0xFFFFFF), 0);
        lv_obj_set_style_text_font(l, _F(16), 0);
        lv_obj_center(l);
        lv_obj_align(g_btn_ok, cancel ? LV_ALIGN_BOTTOM_RIGHT : LV_ALIGN_BOTTOM_MID,
                     cancel ? -30 : 0, -20);
        lv_obj_add_event_cb(g_btn_ok, ok_cb ? ok_cb : _on_dismiss, LV_EVENT_CLICKED, NULL);
    }

    if (cancel) {
        g_btn_cancel = lv_btn_create(g_panel);
        lv_obj_set_size(g_btn_cancel, bw, bh);
        lv_obj_set_style_radius(g_btn_cancel, 8, 0);
        lv_obj_set_style_bg_color(g_btn_cancel, lv_color_hex(0xE0E0E0), 0);
        lv_obj_t *l = lv_label_create(g_btn_cancel);
        lv_label_set_text(l, cancel_txt ? cancel_txt : "Cancel");
        lv_obj_set_style_text_color(l, lv_color_hex(0x333333), 0);
        lv_obj_set_style_text_font(l, _F(16), 0);
        lv_obj_center(l);
        lv_obj_align(g_btn_cancel, LV_ALIGN_BOTTOM_LEFT, 30, -20);
        lv_obj_add_event_cb(g_btn_cancel, _on_cancel, LV_EVENT_CLICKED, NULL);
    }
}

static void _hide_all(void)
{
    if (g_bar)     lv_obj_add_flag(g_bar,    LV_OBJ_FLAG_HIDDEN);
    if (g_status)  lv_obj_add_flag(g_status, LV_OBJ_FLAG_HIDDEN);
    if (g_btn_ok)  { lv_obj_del(g_btn_ok);   g_btn_ok   = NULL; }
    if (g_btn_cancel) { lv_obj_del(g_btn_cancel); g_btn_cancel = NULL; }
    if (g_overlay) lv_obj_add_flag(g_overlay, LV_OBJ_FLAG_HIDDEN);
}

/* ================================================================================
  *  PUBLIC API
  * ================================================================================ */

void ota_ui_init(void) { _ensure_created(); }

void ota_ui_show_checking(void)
{
    _ensure_created();
    lv_label_set_text(g_title, "更新升级");
    lv_obj_clear_flag(g_title,  LV_OBJ_FLAG_HIDDEN);
    lv_obj_add_flag(g_bar,      LV_OBJ_FLAG_HIDDEN);
    lv_obj_clear_flag(g_status, LV_OBJ_FLAG_HIDDEN);
    lv_obj_add_flag(g_info,     LV_OBJ_FLAG_HIDDEN);
    lv_label_set_text(g_status, "正在检查更新...");
    lv_obj_set_style_text_color(g_status, lv_color_hex(0x888888), 0);
    if (g_btn_ok)     { lv_obj_del(g_btn_ok);     g_btn_ok     = NULL; }
    if (g_btn_cancel) { lv_obj_del(g_btn_cancel); g_btn_cancel = NULL; }
    lv_obj_clear_flag(g_overlay, LV_OBJ_FLAG_HIDDEN);
    lv_obj_move_foreground(g_overlay);
}

/* info / error — "确定"按钮只是关闭，不触发升级 */
```

```c
void ota_ui_show_info(const char *title, const char *msg)
{
    _ensure_created();
    lv_label_set_text(g_title, title ? title : "");
    lv_obj_clear_flag(g_title,  LV_OBJ_FLAG_HIDDEN);
    lv_obj_add_flag(g_bar,      LV_OBJ_FLAG_HIDDEN);
    lv_obj_clear_flag(g_status, LV_OBJ_FLAG_HIDDEN);
    lv_obj_add_flag(g_info,     LV_OBJ_FLAG_HIDDEN);
    lv_label_set_text(g_status, msg ? msg : "");
    lv_obj_set_style_text_color(g_status, lv_color_hex(0x555555), 0);
    _show_buttons(1, 0, "确定", NULL, _on_dismiss);  /* dismiss, NOT confirm */
    lv_obj_clear_flag(g_overlay, LV_OBJ_FLAG_HIDDEN);
    lv_obj_move_foreground(g_overlay);
}

void ota_ui_show_error(const char *msg)
{
    _ensure_created();
    lv_label_set_text(g_title, "升级失败");
    lv_obj_clear_flag(g_title, LV_OBJ_FLAG_HIDDEN);
    lv_obj_add_flag(g_bar,    LV_OBJ_FLAG_HIDDEN);
    if (msg && g_status) {
        lv_label_set_text(g_status, msg);
        lv_obj_set_style_text_color(g_status, lv_color_hex(0xE53935), 0);
        lv_obj_clear_flag(g_status, LV_OBJ_FLAG_HIDDEN);
    }
    lv_obj_add_flag(g_info, LV_OBJ_FLAG_HIDDEN);
    _show_buttons(1, 0, "确定", NULL, _on_dismiss);  /* dismiss, NOT confirm */
    lv_obj_clear_flag(g_overlay, LV_OBJ_FLAG_HIDDEN);
    lv_obj_move_foreground(g_overlay);
}

/* upgrade popup — "立即升级" → _on_confirm, "稍后" → _on_cancel */
void ota_ui_show_popup(const char *cur_ver, const char *new_ver, int size_kb)
{
    _ensure_created();
    lv_obj_add_flag(g_bar,    LV_OBJ_FLAG_HIDDEN);
    lv_obj_add_flag(g_status, LV_OBJ_FLAG_HIDDEN);
    lv_label_set_text(g_title, "发现新固件");
    lv_obj_clear_flag(g_title, LV_OBJ_FLAG_HIDDEN);
    lv_obj_clear_flag(g_info,  LV_OBJ_FLAG_HIDDEN);

    char buf[256];
    float sz_mb = (float)size_kb / 1024.0f;
    snprintf(buf, sizeof(buf),
             "当前版本: %s\n"
             "新版本:   %s\n"
             "大小:     %.1f MB\n\n"
             "是否立即升级?",
             cur_ver, new_ver, sz_mb);
    lv_label_set_text(g_info, buf);

    _show_buttons(1, 1, "立即升级", "稍后", _on_confirm);
    lv_obj_clear_flag(g_overlay, LV_OBJ_FLAG_HIDDEN);
    lv_obj_move_foreground(g_overlay);
}

/* downloading — show "取消" button that can abort */
void ota_ui_show_downloading(void)
{
    _ensure_created();
    lv_label_set_text(g_title, "正在升级固件");
    lv_obj_clear_flag(g_title,  LV_OBJ_FLAG_HIDDEN);
```

```
    lv_obj_add_flag(g_info,     LV_OBJ_FLAG_HIDDEN);
    lv_obj_clear_flag(g_bar,    LV_OBJ_FLAG_HIDDEN);
    lv_obj_clear_flag(g_status, LV_OBJ_FLAG_HIDDEN);
    lv_bar_set_value(g_bar, 0, LV_ANIM_OFF);
    lv_label_set_text(g_status, "正在准备下载...");
    lv_obj_set_style_text_color(g_status, lv_color_hex(0x888888), 0);

    /* abort button — just hide overlay */
    if (g_btn_ok)     { lv_obj_del(g_btn_ok);     g_btn_ok     = NULL; }
    if (g_btn_cancel) { lv_obj_del(g_btn_cancel); g_btn_cancel = NULL; }
    g_btn_cancel = lv_btn_create(g_panel);
    lv_obj_set_size(g_btn_cancel, 80, 32);
    lv_obj_set_style_radius(g_btn_cancel, 8, 0);
    lv_obj_set_style_bg_color(g_btn_cancel, lv_color_hex(0xE0E0E0), 0);
    lv_obj_t *l = lv_label_create(g_btn_cancel);
    lv_label_set_text(l, "取消");
    lv_obj_set_style_text_color(l, lv_color_hex(0x333333), 0);
    lv_obj_set_style_text_font(l, _F(14), 0);
    lv_obj_center(l);
    lv_obj_align(g_btn_cancel, LV_ALIGN_BOTTOM_MID, 0, -20);
    lv_obj_add_event_cb(g_btn_cancel, _on_cancel, LV_EVENT_CLICKED, NULL);

    lv_obj_clear_flag(g_overlay, LV_OBJ_FLAG_HIDDEN);
    lv_obj_move_foreground(g_overlay);
}

void ota_ui_update_progress(int pct, const char *st)
{
    if (!g_overlay || lv_obj_has_flag(g_overlay, LV_OBJ_FLAG_HIDDEN)) return;
    if (pct < 0) pct = 0;
    if (pct > 100) pct = 100;
    lv_bar_set_value(g_bar, pct, LV_ANIM_OFF);
    if (st && g_status) lv_label_set_text(g_status, st);
}

void ota_ui_hide(void) { _hide_all(); }

void ota_ui_set_confirm_cb(ota_ui_btn_cb_t cb)  { g_confirm_cb = cb; }
void ota_ui_set_cancel_cb(ota_ui_btn_cb_t cb)   { g_cancel_cb  = cb; }

/*
 * time_sync.c - background NTP synchronization for hardware RTC.
 *
 * Flow:
 *   startup thread -> watch network up transitions -> query NTP -> write RTC
 *
 * The simulator keeps this module as a no-op so Windows builds stay simple.
 */

#include "time_sync.h"

#include <stdio.h>
#include <string.h>

#define TIME_SYNC_NTP_PORT        123
#define TIME_SYNC_NTP_DELTA       2208988800UL
#define TIME_SYNC_RETRY_MS        30000
#define TIME_SYNC_FIRST_DELAY_MS  3000
#define TIME_SYNC_POLL_MS         1000
#define TIME_SYNC_SOCKET_TO_MS    3000
#define TIME_SYNC_VALID_YEAR      2024

static volatile bool g_time_synced = false;
```

```
static volatile bool g_sync_requested = false;
static char g_status[96] = "time sync not started";

static void _set_status(const char *s)
{
    if (!s)
        return;
    strncpy(g_status, s, sizeof(g_status) - 1);
    g_status[sizeof(g_status) - 1] = '\0';
}

bool time_sync_is_synced(void)
{
    return g_time_synced;
}

bool time_sync_rtc_time_looks_valid(time_t now)
{
    struct tm *tm_info = localtime(&now);
    if (!tm_info)
        return false;
    return (tm_info->tm_year + 1900) >= TIME_SYNC_VALID_YEAR;
}

const char *time_sync_status_text(void)
{
    return g_status;
}

#if defined(RTTHREAD_VERSION) || defined(__RTTHREAD__) || defined(RT_USING_LWIP)

#include <rtthread.h>
#include <rtdevice.h>
#include <sys/socket.h>
#include <netdb.h>
#include <sys/time.h>
#include <unistd.h>
#include "lwip/netif.h"
#include "lwip/ip4_addr.h"

static const char *const s_ntp_hosts[] = {
    "ntp.aliyun.com",
    "ntp.tencent.com",
    "cn.pool.ntp.org",
    "time.windows.com",
};

static rt_thread_t g_sync_thread = RT_NULL;

static bool _network_ready(void)
{
    struct netif *netif;

    for (netif = netif_list; netif; netif = netif->next) {
        if (netif_is_up(netif) &&
            netif_is_link_up(netif) &&
            !ip4_addr_isany_val(*netif_ip4_addr(netif))) {
            return true;
        }
    }

    return false;
}
```

```
static void _delay_with_network_watch(uint32_t delay_ms, bool *was_ready)
{
    uint32_t elapsed = 0;

    while (elapsed < delay_ms) {
        bool ready;

        rt_thread_mdelay(TIME_SYNC_POLL_MS);
        elapsed += TIME_SYNC_POLL_MS;

        ready = _network_ready();
        if (ready && !*was_ready) {
            g_sync_requested = true;
            _set_status("network up, NTP requested");
            *was_ready = true;
            break;
        }

        *was_ready = ready;
        if (!ready)
            break;
    }
}

static uint32_t _read_be32(const uint8_t *p)
{
    return ((uint32_t)p[0] << 24) |
           ((uint32_t)p[1] << 16) |
           ((uint32_t)p[2] << 8) |
           ((uint32_t)p[3]);
}

static int _query_ntp_once(const char *host, time_t *local_time_out)
{
    int sock = -1;
    struct hostent *he;
    struct sockaddr_in addr;
    struct timeval tv;
    uint8_t packet[48];
    int n;
    uint32_t ntp_seconds;
    time_t utc_time;

    he = gethostbyname(host);
    if (!he || !he->h_addr_list || !he->h_addr_list[0]) {
        snprintf(g_status, sizeof(g_status), "NTP DNS failed: %s", host);
        return -1;
    }

    sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) {
        _set_status("NTP socket failed");
        return -1;
    }

    tv.tv_sec = TIME_SYNC_SOCKET_TO_MS / 1000;
    tv.tv_usec = (TIME_SYNC_SOCKET_TO_MS % 1000) * 1000;
    setsockopt(sock, SOL_SOCKET, SO_RCVTIMEO, &tv, sizeof(tv));

    memset(&addr, 0, sizeof(addr));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(TIME_SYNC_NTP_PORT);
```

```
    memcpy(&addr.sin_addr, he->h_addr_list[0], sizeof(addr.sin_addr));

    memset(packet, 0, sizeof(packet));
    packet[0] = 0x1B; /* LI=0, VN=3, mode=client */

    if (sendto(sock, packet, sizeof(packet), 0,
               (struct sockaddr *)&addr, sizeof(addr)) < 0) {
        close(sock);
        snprintf(g_status, sizeof(g_status), "NTP send failed: %s", host);
        return -1;
    }

    n = recvfrom(sock, packet, sizeof(packet), 0, NULL, NULL);
    close(sock);

    if (n < 48) {
        snprintf(g_status, sizeof(g_status), "NTP timeout: %s", host);
        return -1;
    }

    ntp_seconds = _read_be32(&packet[40]);
    if (ntp_seconds <= TIME_SYNC_NTP_DELTA) {
        snprintf(g_status, sizeof(g_status), "NTP invalid time: %s", host);
        return -1;
    }

    utc_time = (time_t)(ntp_seconds - TIME_SYNC_NTP_DELTA);
    *local_time_out = utc_time + TIME_SYNC_UTC_OFFSET_SECONDS;
    snprintf(g_status, sizeof(g_status), "NTP OK: %s", host);
    return 0;
}

static int _write_rtc(time_t local_time)
{
    rt_device_t rtc;
    struct timeval tv;
    const char *names[] = { "rtc", "rtc0" };

    tv.tv_sec = local_time;
    tv.tv_usec = 0;
    settimeofday(&tv, NULL);

    for (unsigned i = 0; i < sizeof(names) / sizeof(names[0]); i++) {
        rtc = rt_device_find(names[i]);
        if (!rtc)
            continue;

        rt_device_open(rtc, 0);
        if (rt_device_control(rtc, RT_DEVICE_CTRL_RTC_SET_TIME, &local_time) == RT_EOK) {
            snprintf(g_status, sizeof(g_status), "RTC synced by NTP");
            return 0;
        }
    }

    _set_status("RTC device not found or set failed");
    return -1;
}

static void _sync_thread_entry(void *arg)
{
    bool was_ready = false;

    (void)arg;
```

```
    rt_thread_mdelay(TIME_SYNC_FIRST_DELAY_MS);
    g_sync_requested = true;

    while (1) {
        bool ready = _network_ready();
        time_t now = time(NULL);
        time_t ntp_time = 0;
        bool ok = false;

        if (ready && !was_ready) {
            g_sync_requested = true;
            _set_status("network up, NTP requested");
        }
        was_ready = ready;

        if (!ready) {
            _set_status("time sync waiting for network");
            rt_thread_mdelay(TIME_SYNC_POLL_MS);
            continue;
        }

        if (!g_sync_requested) {
            rt_thread_mdelay(TIME_SYNC_POLL_MS);
            continue;
        }

        g_sync_requested = false;

        if (time_sync_rtc_time_looks_valid(now))
            _set_status("RTC valid, correcting by NTP");
        else
            _set_status("RTC invalid, correcting by NTP");

        for (unsigned i = 0; i < sizeof(s_ntp_hosts) / sizeof(s_ntp_hosts[0]); i++) {
            if (_query_ntp_once(s_ntp_hosts[i], &ntp_time) == 0) {
                if (_write_rtc(ntp_time) == 0) {
                    g_time_synced = true;
                    ok = true;
                    break;
                }
            }
        }

        if (!ok) {
            g_sync_requested = true;
            _delay_with_network_watch(TIME_SYNC_RETRY_MS, &was_ready);
        }
    }
}

void time_sync_request_sync(void)
{
    g_sync_requested = true;
    _set_status("NTP sync requested");
}

void time_sync_start_auto(void)
{
    if (g_sync_thread) {
        time_sync_request_sync();
        return;
    }
```

```
    _set_status("time sync waiting for network");
    g_sync_thread = rt_thread_create("ntp_sync",
                                     _sync_thread_entry,
                                     RT_NULL,
                                     4096,
                                     22,
                                     10);
    if (g_sync_thread)
        rt_thread_startup(g_sync_thread);
    else
        _set_status("time sync thread create failed");
}

#else

void time_sync_request_sync(void)
{
    _set_status("time sync simulator no-op");
}

void time_sync_start_auto(void)
{
    _set_status("time sync simulator no-op");
}

#endif

/*
 * license_auth.c - offline activation code with HMAC-SHA256.
 *
 * Code format:
 *   A1-<EXPIRY_HEX>-<NONCE_HEX>-<SIG_HEX16>
 *
 * EXPIRY_HEX is a Unix timestamp in seconds.  NONCE_HEX is public random
 * data from the admin tool.  SIG_HEX16 is the first 64 bits of:
 *   HMAC-SHA256(secret_key, device_id | expiry | nonce | product | version)
 */

#include "license_auth.h"

#include <ctype.h>
#include <stdio.h>
#include <string.h>

#define LICENSE_PRODUCT_ID "A1"
#define LICENSE_CODE_VERSION "v2"

/*
 * Change this key before release and keep the same value in tools/license_gen.py.
 * For stronger protection against firmware extraction, move to public-key
 * signatures later so the device only stores a public key.
 */
static const uint8_t s_license_key[] =
    "A1:2026:factory-license-key:replace-with-random-32-bytes";

static const char *const s_license_paths[] = {
    "data/license_auth.dat",
    "../data/license_auth.dat",
    "license_auth.dat",
};

typedef struct {
```

```
    uint8_t data[64];
    uint32_t datalen;
    uint64_t bitlen;
    uint32_t state[8];
} sha256_ctx_t;

static const uint32_t k256[64] = {
    0x428a2f98,0x71374491,0xb5c0fbcf,0xe9b5dba5,
    0x3956c25b,0x59f111f1,0x923f82a4,0xab1c5ed5,
    0xd807aa98,0x12835b01,0x243185be,0x550c7dc3,
    0x72be5d74,0x80deb1fe,0x9bdc06a7,0xc19bf174,
    0xe49b69c1,0xefbe4786,0x0fc19dc6,0x240ca1cc,
    0x2de92c6f,0x4a7484aa,0x5cb0a9dc,0x76f988da,
    0x983e5152,0xa831c66d,0xb00327c8,0xbf597fc7,
    0xc6e00bf3,0xd5a79147,0x06ca6351,0x14292967,
    0x27b70a85,0x2e1b2138,0x4d2c6dfc,0x53380d13,
    0x650a7354,0x766a0abb,0x81c2c92e,0x92722c85,
    0xa2bfe8a1,0xa81a664b,0xc24b8b70,0xc76c51a3,
    0xd192e819,0xd6990624,0xf40e3585,0x106aa070,
    0x19a4c116,0x1e376c08,0x2748774c,0x34b0bcb5,
    0x391c0cb3,0x4ed8aa4a,0x5b9cca4f,0x682e6ff3,
    0x748f82ee,0x78a5636f,0x84c87814,0x8cc70208,
    0x90befffa,0xa4506ceb,0xbef9a3f7,0xc67178f2
};

static uint32_t _rotr(uint32_t x, uint32_t n)
{
    return (x >> n) | (x << (32 - n));
}

static void _sha256_transform(sha256_ctx_t *ctx, const uint8_t data[])
{
    uint32_t a, b, c, d, e, f, g, h;
    uint32_t m[64];

    for (int i = 0, j = 0; i < 16; i++, j += 4) {
        m[i] = ((uint32_t)data[j] << 24) |
               ((uint32_t)data[j + 1] << 16) |
               ((uint32_t)data[j + 2] << 8) |
               ((uint32_t)data[j + 3]);
    }
    for (int i = 16; i < 64; i++) {
        uint32_t s0 = _rotr(m[i - 15], 7) ^ _rotr(m[i - 15], 18) ^ (m[i - 15] >> 3);
        uint32_t s1 = _rotr(m[i - 2], 17) ^ _rotr(m[i - 2], 19) ^ (m[i - 2] >> 10);
        m[i] = m[i - 16] + s0 + m[i - 7] + s1;
    }

    a = ctx->state[0]; b = ctx->state[1]; c = ctx->state[2]; d = ctx->state[3];
    e = ctx->state[4]; f = ctx->state[5]; g = ctx->state[6]; h = ctx->state[7];

    for (int i = 0; i < 64; i++) {
        uint32_t s1 = _rotr(e, 6) ^ _rotr(e, 11) ^ _rotr(e, 25);
        uint32_t ch = (e & f) ^ ((~e) & g);
        uint32_t t1 = h + s1 + ch + k256[i] + m[i];
        uint32_t s0 = _rotr(a, 2) ^ _rotr(a, 13) ^ _rotr(a, 22);
        uint32_t maj = (a & b) ^ (a & c) ^ (b & c);
        uint32_t t2 = s0 + maj;
        h = g;
        g = f;
        f = e;
        e = d + t1;
        d = c;
        c = b;
```

```
        b = a;
        a = t1 + t2;
    }

    ctx->state[0] += a; ctx->state[1] += b; ctx->state[2] += c; ctx->state[3] += d;
    ctx->state[4] += e; ctx->state[5] += f; ctx->state[6] += g; ctx->state[7] += h;
}

static void _sha256_init(sha256_ctx_t *ctx)
{
    ctx->datalen = 0;
    ctx->bitlen = 0;
    ctx->state[0] = 0x6a09e667;
    ctx->state[1] = 0xbb67ae85;
    ctx->state[2] = 0x3c6ef372;
    ctx->state[3] = 0xa54ff53a;
    ctx->state[4] = 0x510e527f;
    ctx->state[5] = 0x9b05688c;
    ctx->state[6] = 0x1f83d9ab;
    ctx->state[7] = 0x5be0cd19;
}

static void _sha256_update(sha256_ctx_t *ctx, const uint8_t *data, size_t len)
{
    for (size_t i = 0; i < len; i++) {
        ctx->data[ctx->datalen++] = data[i];
        if (ctx->datalen == 64) {
            _sha256_transform(ctx, ctx->data);
            ctx->bitlen += 512;
            ctx->datalen = 0;
        }
    }
}

static void _sha256_final(sha256_ctx_t *ctx, uint8_t hash[32])
{
    uint32_t i = ctx->datalen;

    ctx->data[i++] = 0x80;
    if (i > 56) {
        while (i < 64)
            ctx->data[i++] = 0x00;
        _sha256_transform(ctx, ctx->data);
        i = 0;
    }
    while (i < 56)
        ctx->data[i++] = 0x00;

    ctx->bitlen += (uint64_t)ctx->datalen * 8;
    ctx->data[63] = (uint8_t)(ctx->bitlen);
    ctx->data[62] = (uint8_t)(ctx->bitlen >> 8);
    ctx->data[61] = (uint8_t)(ctx->bitlen >> 16);
    ctx->data[60] = (uint8_t)(ctx->bitlen >> 24);
    ctx->data[59] = (uint8_t)(ctx->bitlen >> 32);
    ctx->data[58] = (uint8_t)(ctx->bitlen >> 40);
    ctx->data[57] = (uint8_t)(ctx->bitlen >> 48);
    ctx->data[56] = (uint8_t)(ctx->bitlen >> 56);
    _sha256_transform(ctx, ctx->data);

    for (i = 0; i < 4; i++) {
        hash[i]      = (uint8_t)((ctx->state[0] >> (24 - i * 8)) & 0xFF);
        hash[i + 4]  = (uint8_t)((ctx->state[1] >> (24 - i * 8)) & 0xFF);
        hash[i + 8]  = (uint8_t)((ctx->state[2] >> (24 - i * 8)) & 0xFF);
```

```
        hash[i + 12] = (uint8_t)((ctx->state[3] >> (24 - i * 8)) & 0xFF);
        hash[i + 16] = (uint8_t)((ctx->state[4] >> (24 - i * 8)) & 0xFF);
        hash[i + 20] = (uint8_t)((ctx->state[5] >> (24 - i * 8)) & 0xFF);
        hash[i + 24] = (uint8_t)((ctx->state[6] >> (24 - i * 8)) & 0xFF);
        hash[i + 28] = (uint8_t)((ctx->state[7] >> (24 - i * 8)) & 0xFF);
    }
}

static void _hmac_sha256(const uint8_t *key, size_t key_len,
                         const uint8_t *msg, size_t msg_len,
                         uint8_t out[32])
{
    uint8_t key_block[64];
    uint8_t ipad[64];
    uint8_t opad[64];
    uint8_t inner[32];
    sha256_ctx_t ctx;

    memset(key_block, 0, sizeof(key_block));
    if (key_len > 64) {
        _sha256_init(&ctx);
        _sha256_update(&ctx, key, key_len);
        _sha256_final(&ctx, key_block);
    } else {
        memcpy(key_block, key, key_len);
    }

    for (int i = 0; i < 64; i++) {
        ipad[i] = key_block[i] ^ 0x36;
        opad[i] = key_block[i] ^ 0x5c;
    }

    _sha256_init(&ctx);
    _sha256_update(&ctx, ipad, sizeof(ipad));
    _sha256_update(&ctx, msg, msg_len);
    _sha256_final(&ctx, inner);

    _sha256_init(&ctx);
    _sha256_update(&ctx, opad, sizeof(opad));
    _sha256_update(&ctx, inner, sizeof(inner));
    _sha256_final(&ctx, out);
}

static void _normalize_code(const char *in, char *out, int out_len)
{
    int p = 0;

    if (out_len <= 0)
        return;

    for (int i = 0; in && in[i] && p < out_len - 1; i++) {
        unsigned char c = (unsigned char)in[i];
        if (isspace(c))
            continue;
        out[p++] = (char)toupper(c);
    }
    out[p] = '\0';
}

static bool _is_hex16(const char *s)
{
    if (!s || strlen(s) != 16)
        return false;
```

```
    for (int i = 0; i < 16; i++)
        if (!isxdigit((unsigned char)s[i]))
            return false;
    return true;
}

static uint32_t _default_nonce(time_t expiry)
{
    uint32_t n = 0x9E3779B9u ^ (uint32_t)expiry ^ (uint32_t)((uint64_t)expiry >> 32);
    const char *dev = license_auth_device_id();

    for (int i = 0; dev[i]; i++)
        n = (n * 16777619u) ^ (uint8_t)dev[i];
    return n;
}

static void _make_sig(time_t expiry, uint32_t nonce, char out[17])
{
    char msg[128];
    uint8_t mac[32];

    snprintf(msg, sizeof(msg), "%s|%08llX|%08X|%s|%s",
             license_auth_device_id(),
             (unsigned long long)expiry,
             nonce,
             LICENSE_PRODUCT_ID,
             LICENSE_CODE_VERSION);

    _hmac_sha256(s_license_key, sizeof(s_license_key) - 1,
                 (const uint8_t *)msg, strlen(msg), mac);

    for (int i = 0; i < 8; i++)
        snprintf(out + i * 2, 3, "%02X", mac[i]);
    out[16] = '\0';
}

static bool _save_record(time_t expiry, uint32_t nonce, const char *sig)
{
    for (unsigned i = 0; i < sizeof(s_license_paths) / sizeof(s_license_paths[0]); i++) {
        FILE *fp = fopen(s_license_paths[i], "w");
        if (!fp)
            continue;
        fprintf(fp, "%lld %08X %s\n", (long long)expiry, nonce, sig);
        fclose(fp);
        return true;
    }
    return false;
}

static bool _load_record(time_t *expiry_out)
{
    long long expiry_ll = 0;
    unsigned nonce = 0;
    char sig[17];
    char expected[17];

    for (unsigned i = 0; i < sizeof(s_license_paths) / sizeof(s_license_paths[0]); i++) {
        FILE *fp = fopen(s_license_paths[i], "r");
        if (!fp)
            continue;
        int n = fscanf(fp, "%lld %x %16s", &expiry_ll, &nonce, sig);
        fclose(fp);
```

```
        if (n == 3) {
            _normalize_code(sig, sig, sizeof(sig));
            _make_sig((time_t)expiry_ll, (uint32_t)nonce, expected);
            if (strcmp(sig, expected) == 0) {
                if (expiry_out)
                    *expiry_out = (time_t)expiry_ll;
                return true;
            }
        }
    }
    return false;
}

time_t license_auth_now(void)
{
    return time(NULL);
}

const char *license_auth_device_id(void)
{
    return LICENSE_DEVICE_ID;
}

void license_auth_format_datetime(time_t t, char *out, int out_len)
{
    struct tm *tm_info;

    if (!out || out_len <= 0)
        return;

    tm_info = localtime(&t);
    if (!tm_info) {
        snprintf(out, out_len, "1970-01-01 00:00:00");
        return;
    }

    snprintf(out, out_len, "%04d-%02d-%02d %02d:%02d:%02d",
             tm_info->tm_year + 1900,
             tm_info->tm_mon + 1,
             tm_info->tm_mday,
             tm_info->tm_hour,
             tm_info->tm_min,
             tm_info->tm_sec);
}

void license_auth_make_code(time_t expiry, char *out, int out_len)
{
    uint32_t nonce = _default_nonce(expiry);
    char sig[17];

    if (!out || out_len <= 0)
        return;

    _make_sig(expiry, nonce, sig);
    snprintf(out, out_len, "A1-%08llX-%08X-%s",
             (unsigned long long)expiry,
             nonce,
             sig);
}

void license_auth_make_30day_code(time_t now, char *out, int out_len,
                                  time_t *expiry_out)
{
```

```
    time_t expiry = now + (time_t)LICENSE_AUTH_TRIAL_DAYS * LICENSE_AUTH_SECONDS_PER_DAY;

    if (expiry_out)
        *expiry_out = expiry;
    license_auth_make_code(expiry, out, out_len);
}

license_auth_result_t license_auth_activate(const char *code,
                                            time_t now,
                                            time_t *expiry_out)
{
    char normalized[LICENSE_AUTH_CODE_MAX];
    char sig[17];
    char expected[17];
    unsigned long long expiry_ull = 0;
    unsigned nonce = 0;
    time_t expiry;

    _normalize_code(code, normalized, sizeof(normalized));

    if (sscanf(normalized, "A1-%llX-%X-%16s", &expiry_ull, &nonce, sig) != 3)
        return LICENSE_AUTH_BAD_FORMAT;

    _normalize_code(sig, sig, sizeof(sig));
    if (!_is_hex16(sig))
        return LICENSE_AUTH_BAD_FORMAT;

    expiry = (time_t)expiry_ull;
    _make_sig(expiry, (uint32_t)nonce, expected);
    if (strcmp(sig, expected) != 0)
        return LICENSE_AUTH_BAD_MAC;
    if (expiry <= now)
        return LICENSE_AUTH_EXPIRED;
    if (!_save_record(expiry, (uint32_t)nonce, sig))
        return LICENSE_AUTH_SAVE_FAILED;

    if (expiry_out)
        *expiry_out = expiry;
    return LICENSE_AUTH_OK;
}

bool license_auth_is_valid(time_t now, time_t *expiry_out)
{
    time_t expiry = 0;

    if (!_load_record(&expiry))
        return false;
    if (expiry <= now)
        return false;

    if (expiry_out)
        *expiry_out = expiry;
    return true;
}

const char *license_auth_result_text(license_auth_result_t r)
{
    switch (r) {
    case LICENSE_AUTH_OK:
        return "授权成功";
    case LICENSE_AUTH_BAD_FORMAT:
        return "激活码格式错误";
    case LICENSE_AUTH_BAD_MAC:
```

```
        return "激活码校验失败";
    case LICENSE_AUTH_EXPIRED:
        return "激活码已过期";
    case LICENSE_AUTH_SAVE_FAILED:
        return "授权保存失败";
    default:
        return "授权失败";
    }
}

/*
 * login_screen.c - dynamic startup authorization page.
 */

#include "login_screen.h"
#include "license_auth.h"

#include "aic_lang.h"
#include "aic_ui.h"
#include "lvgl.h"

#include <stdio.h>
#include <string.h>

static lv_obj_t *g_root;
static lv_obj_t *g_clock_label;
static lv_obj_t *g_device_label;
static lv_obj_t *g_status_label;
static lv_obj_t *g_expiry_label;
static lv_obj_t *g_demo_label;
static lv_obj_t *g_code_ta;
static lv_obj_t *g_action_btn;
static lv_obj_t *g_action_label;
static lv_obj_t *g_keyboard;
static lv_timer_t *g_clock_timer;
static login_screen_success_cb_t g_success_cb;
static char g_demo_code[LICENSE_AUTH_CODE_MAX];
static time_t g_demo_expiry;
static bool g_has_valid_license;

static void _refresh_status(void);

static void _set_font(lv_obj_t *obj, int size)
{
    lv_obj_set_style_text_font(obj, aic_get_multi_lang_font(size),
                               LV_PART_MAIN | LV_STATE_DEFAULT);
}

static lv_obj_t *_make_label(lv_obj_t *parent, const char *text,
                             int x, int y, int w, int h, int size,
                             lv_color_t color)
{
    lv_obj_t *label = lv_label_create(parent);
    lv_obj_set_pos(label, x, y);
    lv_obj_set_size(label, w, h);
    lv_label_set_text(label, text);
    _set_font(label, size);
    lv_obj_set_style_text_color(label, color, LV_PART_MAIN | LV_STATE_DEFAULT);
    return label;
}

static void _textarea_focused(lv_event_t *e)
{
```

```
    lv_obj_t *ta = lv_event_get_target(e);
    lv_keyboard_set_textarea(g_keyboard, ta);
    lv_obj_clear_flag(g_keyboard, LV_OBJ_FLAG_HIDDEN);
}

static void _textarea_defocused(lv_event_t *e)
{
    (void)e;
    lv_keyboard_set_textarea(g_keyboard, NULL);
    lv_obj_add_flag(g_keyboard, LV_OBJ_FLAG_HIDDEN);
}

static void _keyboard_ready(lv_event_t *e)
{
    (void)e;
    lv_keyboard_set_textarea(g_keyboard, NULL);
    lv_obj_add_flag(g_keyboard, LV_OBJ_FLAG_HIDDEN);
}

static void _enter_main(void)
{
    lv_obj_t *old_root = g_root;

    if (g_clock_timer) {
        lv_timer_delete(g_clock_timer);
        g_clock_timer = NULL;
    }

    if (g_success_cb)
        g_success_cb();

    if (old_root)
        lv_obj_delete(old_root);

    g_root = NULL;
}

static void _action_clicked(lv_event_t *e)
{
    (void)e;

    if (g_has_valid_license) {
        _enter_main();
        return;
    }

    const char *code = lv_textarea_get_text(g_code_ta);
    time_t now = license_auth_now();
    time_t expiry = 0;
    license_auth_result_t ret = license_auth_activate(code, now, &expiry);

    if (ret == LICENSE_AUTH_OK) {
        char expiry_buf[32];
        license_auth_format_datetime(expiry, expiry_buf, sizeof(expiry_buf));
        lv_label_set_text_fmt(g_status_label, "授权成功，可使用至 %s", expiry_buf);
        lv_label_set_text(g_action_label, "进入系统");
        g_has_valid_license = true;
    } else {
        lv_label_set_text(g_status_label, license_auth_result_text(ret));
    }

    _refresh_status();
}
```

```
static void _refresh_clock(lv_timer_t *t)
{
    char now_buf[32];
    time_t now = license_auth_now();

    (void)t;
    license_auth_format_datetime(now, now_buf, sizeof(now_buf));
    lv_label_set_text_fmt(g_clock_label, "RTC时间  %s", now_buf);
    _refresh_status();
}

static void _refresh_status(void)
{
    char expiry_buf[32];
    char demo_expiry_buf[32];
    time_t now = license_auth_now();
    time_t expiry = 0;

    g_has_valid_license = license_auth_is_valid(now, &expiry);

    if (g_has_valid_license) {
        long long remain = (long long)(expiry - now);
        int days = (int)(remain / LICENSE_AUTH_SECONDS_PER_DAY);
        int hours = (int)((remain % LICENSE_AUTH_SECONDS_PER_DAY) / 3600);
        int mins = (int)((remain % 3600) / 60);

        license_auth_format_datetime(expiry, expiry_buf, sizeof(expiry_buf));
        lv_label_set_text_fmt(g_status_label,
                              "授权有效  剩余 %d 天 %02d:%02d",
                              days, hours, mins);
        lv_label_set_text_fmt(g_expiry_label, "到期时间  %s", expiry_buf);
        lv_label_set_text(g_action_label, "进入系统");
    } else {
        lv_label_set_text(g_status_label, "未授权，请输入激活码");
        lv_label_set_text(g_expiry_label, "到期时间  --");
        lv_label_set_text(g_action_label, "激活设备");
    }

    license_auth_format_datetime(g_demo_expiry, demo_expiry_buf, sizeof(demo_expiry_buf));
    lv_label_set_text_fmt(g_demo_label, "测试激活码  %s  截止 %s",
                          g_demo_code, demo_expiry_buf);
}

void login_screen_start(login_screen_success_cb_t success_cb)
{
    lv_obj_t *panel;
    lv_obj_t *title;
    lv_obj_t *hint;

    g_success_cb = success_cb;
    g_demo_code[0] = '\0';
    g_demo_expiry = 0;
    license_auth_make_30day_code(license_auth_now(), g_demo_code,
                                 sizeof(g_demo_code), &g_demo_expiry);

    g_root = lv_obj_create(NULL);
    lv_obj_set_style_bg_color(g_root, lv_color_hex(0xF6F8FB),
                              LV_PART_MAIN | LV_STATE_DEFAULT);
    lv_obj_set_style_bg_opa(g_root, LV_OPA_COVER,
                            LV_PART_MAIN | LV_STATE_DEFAULT);

    title = _make_label(g_root, "设备授权", 0, 36, 1024, 46, 28,
```

```
                    lv_color_hex(0x25324A));
lv_obj_set_style_text_align(title, LV_TEXT_ALIGN_CENTER,
                            LV_PART_MAIN | LV_STATE_DEFAULT);

panel = lv_obj_create(g_root);
lv_obj_set_pos(panel, 242, 104);
lv_obj_set_size(panel, 540, 360);
lv_obj_set_style_radius(panel, 6, LV_PART_MAIN | LV_STATE_DEFAULT);
lv_obj_set_style_bg_color(panel, lv_color_hex(0xFFFFFF),
                          LV_PART_MAIN | LV_STATE_DEFAULT);
lv_obj_set_style_border_color(panel, lv_color_hex(0xD6DCE8),
                              LV_PART_MAIN | LV_STATE_DEFAULT);
lv_obj_set_style_border_width(panel, 1, LV_PART_MAIN | LV_STATE_DEFAULT);
lv_obj_set_style_pad_all(panel, 0, LV_PART_MAIN | LV_STATE_DEFAULT);

g_clock_label = _make_label(panel, "RTC 时间  --", 36, 30, 470, 30, 20,
                            lv_color_hex(0x1E293B));
g_device_label = _make_label(panel, "", 36, 72, 470, 26, 16,
                             lv_color_hex(0x475569));
lv_label_set_text_fmt(g_device_label, "设备编号  %s", license_auth_device_id());

g_status_label = _make_label(panel, "未授权，请输入激活码", 36, 112, 470, 30, 18,
                             lv_color_hex(0xB45309));
g_expiry_label = _make_label(panel, "到期时间  --", 36, 150, 470, 26, 16,
                             lv_color_hex(0x475569));

g_code_ta = lv_textarea_create(panel);
lv_obj_set_pos(g_code_ta, 36, 192);
lv_obj_set_size(g_code_ta, 468, 48);
lv_textarea_set_one_line(g_code_ta, true);
lv_textarea_set_max_length(g_code_ta, LICENSE_AUTH_CODE_MAX - 1);
lv_textarea_set_placeholder_text(g_code_ta, "输入激活码");
_set_font(g_code_ta, 18);
lv_obj_add_event_cb(g_code_ta, _textarea_focused, LV_EVENT_FOCUSED, NULL);
lv_obj_add_event_cb(g_code_ta, _textarea_defocused, LV_EVENT_DEFOCUSED, NULL);

g_action_btn = lv_button_create(panel);
lv_obj_set_pos(g_action_btn, 36, 260);
lv_obj_set_size(g_action_btn, 468, 48);
lv_obj_set_style_radius(g_action_btn, 4, LV_PART_MAIN | LV_STATE_DEFAULT);
lv_obj_set_style_bg_color(g_action_btn, lv_color_hex(0x1E66F5),
                          LV_PART_MAIN | LV_STATE_DEFAULT);
lv_obj_add_event_cb(g_action_btn, _action_clicked, LV_EVENT_CLICKED, NULL);
g_action_label = lv_label_create(g_action_btn);
lv_label_set_text(g_action_label, "激活设备");
_set_font(g_action_label, 18);
lv_obj_center(g_action_label);

hint = _make_label(panel, "激活后有效期为 30 天，授权时间精确到秒。", 36, 324,
                   468, 22, 14, lv_color_hex(0x64748B));
lv_obj_set_style_text_align(hint, LV_TEXT_ALIGN_CENTER,
                            LV_PART_MAIN | LV_STATE_DEFAULT);

g_demo_label = _make_label(g_root, "", 120, 486, 784, 28, 15,
                           lv_color_hex(0x64748B));
lv_obj_set_style_text_align(g_demo_label, LV_TEXT_ALIGN_CENTER,
                            LV_PART_MAIN | LV_STATE_DEFAULT);

g_keyboard = lv_keyboard_create(g_root);
lv_obj_set_size(g_keyboard, 1024, 210);
lv_obj_align(g_keyboard, LV_ALIGN_BOTTOM_MID, 0, 0);
lv_keyboard_set_mode(g_keyboard, LV_KEYBOARD_MODE_TEXT_UPPER);
lv_keyboard_set_popovers(g_keyboard, false);
```

```
    lv_obj_add_flag(g_keyboard, LV_OBJ_FLAG_HIDDEN);
    lv_obj_add_event_cb(g_keyboard, _keyboard_ready, LV_EVENT_READY, NULL);
    lv_obj_add_event_cb(g_keyboard, _keyboard_ready, LV_EVENT_CANCEL, NULL);

    lv_scr_load(g_root);
    _refresh_clock(NULL);
    g_clock_timer = lv_timer_create(_refresh_clock, 1000, NULL);
}
```

```
#!/usr/bin/env python3
"""
Generate A1 offline activation codes.

Examples:
   python tools/license_gen.py --device A1-DEMO-001 --days 30
   python tools/license_gen.py --device A1-DEMO-001 --expiry "2026-08-28 16:30:00"
   python tools/license_gen.py --device A1-DEMO-001 --days 90 --start "2026-07-29 16:30:00"
"""

from __future__ import annotations

import argparse
import datetime as dt
import hmac
import hashlib
import secrets
import sys


PRODUCT_ID = "A1"
CODE_VERSION = "v2"

# Must match s_license_key in ui_builder/custom/license_auth.c.
# Change both places before release.
SECRET_KEY = b"A1:2026:factory-license-key:replace-with-random-32-bytes"


def parse_local_datetime(value: str) -> dt.datetime:
    try:
        return dt.datetime.strptime(value, "%Y-%m-%d %H:%M:%S")
    except ValueError as exc:
        raise argparse.ArgumentTypeError(
            "use format YYYY-MM-DD HH:MM:SS"
        ) from exc


def to_timestamp(local_dt: dt.datetime) -> int:
    return int(local_dt.timestamp())


def format_local(ts: int) -> str:
    return dt.datetime.fromtimestamp(ts).strftime("%Y-%m-%d %H:%M:%S")


def make_signature(device: str, expiry_ts: int, nonce: int) -> str:
    msg = f"{device}|{expiry_ts:08X}|{nonce:08X}|{PRODUCT_ID}|{CODE_VERSION}".encode(
        "ascii"
    )
    digest = hmac.new(SECRET_KEY, msg, hashlib.sha256).digest()
    return digest[:8].hex().upper()


def make_code(device: str, expiry_ts: int, nonce: int | None = None) -> str:
```

```
    if nonce is None:
        nonce = secrets.randbits(32)
    sig = make_signature(device, expiry_ts, nonce)
    return f"A1-{expiry_ts:08X}-{nonce:08X}-{sig}"


def main() -> int:
    parser = argparse.ArgumentParser(description="Generate A1 activation codes")
    parser.add_argument("--device", required=True, help="device id shown on login screen")
    group = parser.add_mutually_exclusive_group(required=True)
    group.add_argument("--days", type=int, help="valid days from start time")
    group.add_argument(
        "--expiry",
        type=parse_local_datetime,
        help='exact expiry time, e.g. "2026-08-28 16:30:00"',
    )
    parser.add_argument(
        "--start",
        type=parse_local_datetime,
        default=None,
        help='start time for --days, default is now, e.g. "2026-07-29 16:30:00"',
    )
    parser.add_argument(
        "--nonce",
        type=lambda s: int(s, 16),
        default=None,
        help="optional 32-bit hex nonce for reproducible output",
    )
    args = parser.parse_args()

    device = args.device.strip()
    if not device:
        parser.error("--device cannot be empty")

    if args.expiry is not None:
        expiry_dt = args.expiry
    else:
        if args.days <= 0:
            parser.error("--days must be positive")
        start_dt = args.start or dt.datetime.now()
        expiry_dt = start_dt + dt.timedelta(days=args.days)

    expiry_ts = to_timestamp(expiry_dt)
    if expiry_ts <= int(dt.datetime.now().timestamp()):
        print("warning: generated code is already expired for this PC time", file=sys.stderr)

    nonce = args.nonce
    if nonce is not None and not (0 <= nonce <= 0xFFFFFFFF):
        parser.error("--nonce must fit in 32 bits")

    code = make_code(device, expiry_ts, nonce)

    print(f"device : {device}")
    print(f"expiry : {format_local(expiry_ts)}")
    print(f"unix   : {expiry_ts}")
    print(f"code   : {code}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())

/*
```

```
 * Custom code — Modbus RTU (read/write, TCP/RS485) + keyboard care
 *
 * Mapping: btn_userdata = SPxx register index (0=SP00 … 18=SP18).
 * g_param[i] has the label/textarea/write_btn/read_btn widgets that are
 * next to each other on screen at the visual row for SPxx.
 *
 * Callbacks are bound ONCE in custom_init with the CORRECT SP index per
 * button (the button's visual position determines which SPxx it belongs
 * to, regardless of the widget's auto-generated number).
 */

#include "custom.h"
#include "ethernet_comm.h"
#include "modbus_rtu.h"
#include "serial_comm.h"
#include "onenet_ota.h"
#include "ota_ui.h"
#include "license_auth.h"
#include "login_screen.h"
#include "time_sync.h"
#ifdef RTTHREAD_VERSION
#include <rtthread.h>
#endif
#include <string.h>
#include <stdlib.h>
#include <stdio.h>
#include <time.h>
#include <stdarg.h>

static void on_btn_start_clicked(lv_event_t *e);
static void _on_ota_confirm(void);
static void _on_ota_timer(lv_timer_t *t);
static void _on_ota_check_timer(lv_timer_t *t);
static void _ota_progress_cb(int percent, const char *status);
static void _ota_abort_cb(void);
static void _on_ota_button_clicked(lv_event_t *e);
#ifdef RTTHREAD_VERSION
static void _ota_dl_thread_entry(void *arg);
#endif
static void on_dropdown_tcp_mode_changed(lv_event_t *e);
static void on_comm_mode_changed(lv_event_t *e);
static void on_switch_serial(lv_event_t *e);
static void on_50ms_timer(lv_timer_t *timer);
static void on_write_clicked(lv_event_t *e);
static void on_write_all_clicked(lv_event_t *e);
static void on_read_clicked(lv_event_t *e);
static void on_read_all_clicked(lv_event_t *e);
static void on_slave_text_changed(lv_event_t *e);
static void on_main_leave_pressed(lv_event_t *e);
static void on_keyboard_ok(lv_event_t *e);
static void on_keyboard_cancel(lv_event_t *e);
static void on_screen2_keyboard_done(lv_event_t *e);
static void on_screen1_keyboard_done(lv_event_t *e);
static void _kb_restore_zero(lv_obj_t *kb);
static void configure_main_keyboard(screen_t *s);
static void configure_screen2_keyboard(screen_2_t *s2);
static void configure_screen1_keyboard(screen_1_t *s);
static uint16_t _io_cached_y_base_addr(void);
static void param_refresh(void);
static void main_state_save(void);
static void main_state_restore(void);
static void screen1_init_runtime(void);
static void screen1_poll(void);
```

```
static void screen1_start_lv(lv_event_t *e);
static void screen1_stop_lv(lv_event_t *e);
static void screen1_clear_lv(lv_event_t *e);
static void screen1_save_lv(lv_event_t *e);
static void screen1_restore_lv(lv_event_t *e);
static void screen1_apply_lv(lv_event_t *e);
static void screen1_capture_from_ui(screen_1_t *s);
static void screen1_update_summary(screen_1_t *s);
static void screen1_back_main_lv(lv_event_t *e);
static void screen1_back_main_async(void *user_data);
static bool _io_comm_connected(void);
static bool _non_standard(void);
static void _io_release_background_request(void);
static void auth_gate_start(void);
static void auth_gate_enter_main(void);
static void auth_gate_watchdog(lv_timer_t *timer);

static bool g_custom_first_done = false;
static lv_obj_t *g_main_bound_screen = NULL;
static bool g_auth_gate_showing = false;
static bool g_auth_gate_boot_prompt_done = false;
static lv_timer_t *g_auth_watchdog_timer = NULL;
static int  g_comm_idx       = 0;
static bool g_is_server_mode = true;
static uint8_t g_cached_slave = 1;
static char g_serial_baud_str[32] = {0};

typedef struct { lv_obj_t *label, *textarea, *write_btn, *read_btn; } param_row_t;
static param_row_t g_param[19];
static bool        g_param_valid = false;

typedef struct {
     bool valid;
     char slave[16];
     char tcp_ip[32];
     char tcp_port[16];
     char send_log[2048];
     char recv_log[2048];
     char param[19][32];
     uint16_t dropdown_com;
     uint16_t dropdown_tcp_mode;
     uint16_t dropdown_2;
     uint16_t dropdown_baud;
     uint16_t dropdown_parity;
     uint16_t dropdown_data_bits;
     uint16_t dropdown_stop_bits;
     uint16_t dropdown_8;
     uint16_t dropdown_crc_mode;
} main_screen_state_t;

static main_screen_state_t g_main_state;
static lv_obj_t *g_io_y_sw[20];
static uint8_t g_io_bits[28];
static uint8_t g_io_x_bits[28];
static int g_io_diag_error_count;

typedef struct {
     bool valid;
     char y_base[16];
     char x_base[16];
     char link_delay_ms[16];
     char self_interval_ms[16];
     char send_log[2048];
```

```
    char recv_log[2048];
    uint16_t diag_sel;
    uint16_t expected_sel;
    uint16_t link_y_sel;
    uint16_t link_x_sel;
    uint8_t y_state[20];
    uint8_t x_state[28];
} io_screen_state_t;

static io_screen_state_t g_io_state;

typedef struct {
    bool enabled;
    char name[32];
    char unit[16];
    int source_type;
    int source_id;
    int decimal;
    int min_v;
    int max_v;
    uint16_t source_sel;
    uint16_t unit_sel;
    uint16_t color_sel;
    uint32_t raw_min;
    uint32_t raw_max;
    uint64_t raw_sum;
    uint32_t raw_count;
    int32_t raw_last;
    double value_min;
    double value_max;
    double value_sum;
    double value_last;
    int last_plot;
} screen1_channel_state_t;

typedef struct {
    bool valid;
    bool running;
    uint32_t sample_interval_ms;
    uint32_t elapsed_ms;
    /* 中文注释：状态栏刷新计数，避免监控中心每 50ms 重绘导致 PC 端卡顿。 */
    uint32_t status_elapsed_ms;
    uint32_t sample_count;
    lv_obj_t *bound_screen;
    lv_obj_t *bound_chart;
    lv_chart_series_t *series[3];
    bool scan_active;
    bool scan_waiting;
    int scan_ch;
    int pending_ch;
    uint8_t pending_slave;
    uint8_t pending_func;
    uint16_t pending_addr;
    uint16_t pending_count;
    uint16_t pending_type;
    bool pending_ns;
    int pending_timeout;
    screen1_channel_state_t ch[3];
} screen1_state_t;

static screen1_state_t g_screen1_state;
static bool g_screen1_bound = false;
/* 中文注释：返回主界面期间暂停监控轮询，防止切屏过程中继续访问旧控件。 */
```

```
static bool g_screen1_leaving = false;
/* 中文注释：防止 button_5 连续点击重复投递返回任务。 */
static bool g_screen1_back_pending = false;

#define RQ_MAX 19
static int   g_rq_count   = 0;
static int   g_rq_list[RQ_MAX];
static int   g_rq_cur     = -1;
static int   g_rq_timeout = 0;
static bool  g_rq_ns      = false;  /* non_standard flag for active request */
static char  g_rq_label[64];
static bool  g_serial_probing = false;
static int   g_probe_timeout  = 0;
static int   g_probe_retries  = 0;
static bool  g_serial_ok      = false;
static bool  g_probe_is_io    = false;
#define PROBE_MAX_RETRIES 5
static int   g_keepalive_cnt    = 0;
static int   g_keepalive_fail   = 0;
#define KEEPALIVE_INTERVAL   40  /* 50ms * 40 = 2s */
#define KEEPALIVE_MAX_FAIL   3
static int   g_tcp_keepalive_cnt  = 0;
static int   g_tcp_keepalive_fail = 0;
/* 中文注释：手动 03/04 解析类型由 dropdown 选择：uint16、int16 或 float。 */
static int   g_manual_03_parse_mode = 0;
static int   g_manual_04_parse_mode = 0;

/* ---- helpers ---- */
static void get_ta(lv_obj_t *ta, char *o, int n) {
     const char *s = lv_textarea_get_text(ta);
     if (s) { strncpy(o, s, n-1); o[n-1] = '\0'; } else o[0] = '\0';
}
static void get_lbl(lv_obj_t *l, char *o, int n) {
     const char *s = lv_label_get_text(l);
     if (s) { strncpy(o, s, n-1); o[n-1] = '\0'; } else o[0] = '\0';
}
static bool valid(lv_obj_t *o) { return o && lv_obj_is_valid(o); }
static bool blank(const char *s) {
     if (!s || !*s)
         return true;
     while (*s == ' ')
         s++;
     return !*s;
}
static bool scr_ok(void) {
     screen_t *s = screen_get(&ui_manager);
     return s->obj && lv_scr_act() == s->obj;
}

/* 授权成功后的回调：返回 UIBuilder 生成的主界面。 */
static void auth_gate_enter_main(void)
{
     screen_t *s = screen_get(&ui_manager);

     g_auth_gate_showing = false;
     if (s && valid(s->obj))
         lv_scr_load(s->obj);
}

/* 每秒检查一次授权有效期；到期后立即退回授权界面。 */
static void auth_gate_watchdog(lv_timer_t *timer)
{
     time_t expiry = 0;
```

```
    (void)timer;
    if (g_auth_gate_showing)
        return;
    if (license_auth_is_valid(license_auth_now(), &expiry))
        return;

    g_auth_gate_showing = true;
    login_screen_start(auth_gate_enter_main);
}

/* 自定义层授权入口：放在 custom.c 里，避免 UIBuilder 覆盖 ui_init.c 后丢失。 */
static void auth_gate_start(void)
{
    time_t expiry = 0;

    if (!g_auth_watchdog_timer)
        g_auth_watchdog_timer = lv_timer_create(auth_gate_watchdog, 1000, NULL);

    if (g_auth_gate_showing)
        return;

    /* 每次程序启动先显示授权页；界面重建后的 custom_init 不重复弹出。 */
    if (!g_auth_gate_boot_prompt_done) {
        g_auth_gate_boot_prompt_done = true;
        g_auth_gate_showing = true;
        login_screen_start(auth_gate_enter_main);
        return;
    }

    if (license_auth_is_valid(license_auth_now(), &expiry))
        return;

    g_auth_gate_showing = true;
    login_screen_start(auth_gate_enter_main);
}

static void cache_slave_from_textarea(lv_obj_t *ta) {
    char sid[16];
    int v;

    if (!valid(ta))
        return;

    get_ta(ta, sid, sizeof(sid));
    v = atoi(sid);
    if (v > 0)
        g_cached_slave = (uint8_t)v;
}
static uint8_t get_slave(void) {
    screen_t *s = screen_get(&ui_manager);
    if (valid(s->textarea_1)) {
        cache_slave_from_textarea(s->textarea_1);
        return g_cached_slave;
    }
    return g_cached_slave;
}
static void log_hex(lv_obj_t *ta, const uint8_t *f, int n, const char *pfx) {
    if (!valid(ta)) return;
    char hex[256]; modbus_frame_to_hex(f, n, hex, sizeof(hex));
    lv_textarea_add_text(ta, pfx); lv_textarea_add_text(ta, hex); lv_textarea_add_text(ta, "\n");
}
```

```
static void _kb_restore_zero(lv_obj_t *kb) {
    lv_obj_t *ta = lv_keyboard_get_textarea(kb);
    if (!valid(ta)) return;
    const char *txt = lv_textarea_get_text(ta);
    if (txt && txt[0] != '\0') return;  /* user typed something, keep it */
    /* empty → restore "0" for SP00-SP18 param textareas */
    param_refresh();
    for (int i = 0; i < 19; i++) {
        if (g_param_valid && g_param[i].textarea == ta) {
            lv_textarea_set_text(ta, "0");
            break;
        }
    }
}

/* ---- kb ---- */
static void configure_main_keyboard(screen_t *s) {
    if (!s || !valid(s->keyboard_1))
        return;

    lv_obj_set_size(s->keyboard_1, 1024, 210);
    lv_obj_align(s->keyboard_1, LV_ALIGN_BOTTOM_MID, 0, 0);
    lv_keyboard_set_mode(s->keyboard_1, LV_KEYBOARD_MODE_TEXT_UPPER);
    lv_keyboard_set_popovers(s->keyboard_1, false);
}

static void configure_screen2_keyboard(screen_2_t *s2) {
    if (!s2 || !valid(s2->keyboard_1))
        return;

    lv_obj_set_size(s2->keyboard_1, 1024, 210);
    lv_obj_align(s2->keyboard_1, LV_ALIGN_BOTTOM_MID, 0, 0);
    lv_keyboard_set_mode(s2->keyboard_1, LV_KEYBOARD_MODE_NUMBER);
    lv_keyboard_set_popovers(s2->keyboard_1, false);

    if (valid(s2->textarea_1))
        lv_textarea_set_accepted_chars(s2->textarea_1, "0123456789");
    if (valid(s2->textarea_35))
        lv_textarea_set_accepted_chars(s2->textarea_35, "0123456789");
    if (valid(s2->textarea_4))
        lv_textarea_set_accepted_chars(s2->textarea_4, "0123456789");
    if (valid(s2->textarea_5))
        lv_textarea_set_accepted_chars(s2->textarea_5, "0123456789");
}

static void screen1_keyboard_focus_lv(lv_event_t *e)
{
    lv_obj_t *ta = lv_event_get_target(e);
    screen_1_t *s = screen_1_get(&ui_manager);

    if (!s || !valid(s->keyboard_1) || !valid(ta))
        return;
    lv_keyboard_set_textarea(s->keyboard_1, ta);
    lv_obj_clear_flag(s->keyboard_1, LV_OBJ_FLAG_HIDDEN);
    lv_obj_move_foreground(s->keyboard_1);
    lv_obj_align(s->keyboard_1, LV_ALIGN_BOTTOM_MID, 0, 0);
}

static void screen1_keyboard_defocus_lv(lv_event_t *e)
{
    screen_1_t *s = screen_1_get(&ui_manager);

    (void)e;
```

```
    if (!s || !valid(s->keyboard_1))
        return;
    lv_keyboard_set_textarea(s->keyboard_1, NULL);
    lv_obj_add_flag(s->keyboard_1, LV_OBJ_FLAG_HIDDEN);
}

static void screen1_bind_keyboard_textarea(lv_obj_t *ta)
{
    if (!valid(ta))
        return;
    lv_textarea_set_one_line(ta, true);
    lv_obj_add_event_cb(ta, screen1_keyboard_focus_lv, LV_EVENT_FOCUSED, NULL);
    lv_obj_add_event_cb(ta, screen1_keyboard_defocus_lv, LV_EVENT_DEFOCUSED, NULL);
}

static void screen1_make_output_textarea(lv_obj_t *ta)
{
    if (!valid(ta))
        return;
    lv_textarea_set_cursor_click_pos(ta, false);
    lv_obj_clear_flag(ta, LV_OBJ_FLAG_CLICK_FOCUSABLE);
}

static void configure_screen1_keyboard(screen_1_t *s)
{
    if (!s || !valid(s->keyboard_1))
        return;

    lv_obj_set_size(s->keyboard_1, 1024, 210);
    lv_obj_align(s->keyboard_1, LV_ALIGN_BOTTOM_MID, 0, 0);
    lv_keyboard_set_mode(s->keyboard_1, LV_KEYBOARD_MODE_TEXT_UPPER);
    lv_keyboard_set_popovers(s->keyboard_1, false);
    lv_obj_add_flag(s->keyboard_1, LV_OBJ_FLAG_HIDDEN);

    screen1_bind_keyboard_textarea(s->textarea_1);
    screen1_bind_keyboard_textarea(s->textarea_10);
    screen1_bind_keyboard_textarea(s->textarea_11);
    screen1_bind_keyboard_textarea(s->textarea_12);
    screen1_bind_keyboard_textarea(s->textarea_13);
    screen1_bind_keyboard_textarea(s->textarea_14);
    screen1_bind_keyboard_textarea(s->textarea_15);
    screen1_bind_keyboard_textarea(s->textarea_16);
    screen1_bind_keyboard_textarea(s->textarea_17);
    screen1_bind_keyboard_textarea(s->textarea_18);
    screen1_bind_keyboard_textarea(s->textarea_19);
    screen1_bind_keyboard_textarea(s->textarea_20);
    screen1_bind_keyboard_textarea(s->textarea_21);
    screen1_bind_keyboard_textarea(s->textarea_22);
    screen1_bind_keyboard_textarea(s->textarea_23);
    screen1_bind_keyboard_textarea(s->textarea_24);

    if (valid(s->textarea_1))
        lv_textarea_set_accepted_chars(s->textarea_1, "0123456789");
    if (valid(s->textarea_13))
        lv_textarea_set_accepted_chars(s->textarea_13, "0123456789");
    if (valid(s->textarea_14))
        lv_textarea_set_accepted_chars(s->textarea_14, "0123456789");
    if (valid(s->textarea_15))
        lv_textarea_set_accepted_chars(s->textarea_15, "0123456789");
    if (valid(s->textarea_16))
        lv_textarea_set_accepted_chars(s->textarea_16, "0123456789");
    if (valid(s->textarea_17))
        lv_textarea_set_accepted_chars(s->textarea_17, "0123456789");
```

```
    if (valid(s->textarea_18))
        lv_textarea_set_accepted_chars(s->textarea_18, "0123456789");
    if (valid(s->textarea_19))
        lv_textarea_set_accepted_chars(s->textarea_19, "-0123456789");
    if (valid(s->textarea_20))
        lv_textarea_set_accepted_chars(s->textarea_20, "-0123456789");
    if (valid(s->textarea_21))
        lv_textarea_set_accepted_chars(s->textarea_21, "-0123456789");
    if (valid(s->textarea_22))
        lv_textarea_set_accepted_chars(s->textarea_22, "-0123456789");
    if (valid(s->textarea_23))
        lv_textarea_set_accepted_chars(s->textarea_23, "-0123456789");
    if (valid(s->textarea_24))
        lv_textarea_set_accepted_chars(s->textarea_24, "-0123456789");

    /* 中文注释：tabview_2 里的统计值是输出显示，不参与输入，避免点击时弹出键盘。 */
    screen1_make_output_textarea(s->textarea_2);
    screen1_make_output_textarea(s->textarea_3);
    screen1_make_output_textarea(s->textarea_4);
    screen1_make_output_textarea(s->textarea_5);
    screen1_make_output_textarea(s->textarea_6);
    screen1_make_output_textarea(s->textarea_7);
    screen1_make_output_textarea(s->textarea_8);
    screen1_make_output_textarea(s->textarea_9);
    screen1_make_output_textarea(s->textarea_25);
    screen1_make_output_textarea(s->textarea_26);
    screen1_make_output_textarea(s->textarea_27);
    screen1_make_output_textarea(s->textarea_28);
    screen1_make_output_textarea(s->textarea_29);
    screen1_make_output_textarea(s->textarea_30);
    screen1_make_output_textarea(s->textarea_31);
}

static void hide_kb(void) { screen_t *s=screen_get(&ui_manager);
if(s->obj&&valid(s->keyboard_1))lv_obj_add_flag(s->keyboard_1,LV_OBJ_FLAG_HIDDEN); }
static void on_slave_text_changed(lv_event_t *e) {
    cache_slave_from_textarea(lv_event_get_target(e));
}
static void on_main_leave_pressed(lv_event_t *e) {
    (void)e;
    main_state_save();
    g_main_bound_screen = NULL;
}
static void on_keyboard_ok(lv_event_t *e) {
    (void)e;
    cache_slave_from_textarea(screen_get(&ui_manager)->textarea_1);
    main_state_save();
    if (valid(screen_get(&ui_manager)->keyboard_1))
        _kb_restore_zero(screen_get(&ui_manager)->keyboard_1);
    hide_kb();
}
static void on_keyboard_cancel(lv_event_t *e) {
    (void)e;
    cache_slave_from_textarea(screen_get(&ui_manager)->textarea_1);
    main_state_save();
    if (valid(screen_get(&ui_manager)->keyboard_1))
        _kb_restore_zero(screen_get(&ui_manager)->keyboard_1);
    hide_kb();
}
static void on_screen2_keyboard_done(lv_event_t *e) {
    lv_obj_t *kb = lv_event_get_target(e);
    if (!valid(kb))
        return;
```

```
    lv_keyboard_set_textarea(kb, NULL);
    lv_obj_add_flag(kb, LV_OBJ_FLAG_HIDDEN);
}

static void on_screen1_keyboard_done(lv_event_t *e) {
    screen_1_t *s = screen_1_get(&ui_manager);
    lv_obj_t *kb = lv_event_get_target(e);

    if (!valid(kb))
        return;
    if (s && s->obj)
        screen1_capture_from_ui(s);
    lv_keyboard_set_textarea(kb, NULL);
    lv_obj_add_flag(kb, LV_OBJ_FLAG_HIDDEN);
}

/* ---- status ---- */
static void set_status(bool on) {
    screen_t *s = screen_get(&ui_manager);
    if(!valid(s->label_5)||!valid(s->led_2)) return;
    if(on){lv_obj_clear_flag(s->label_5,LV_OBJ_FLAG_HIDDEN);lv_obj_add_flag(s->label_6,LV_OBJ_FL
AG_HIDDEN);lv_led_on(s->led_2);lv_led_off(s->led_3);}
    else  {lv_obj_clear_flag(s->label_6,LV_OBJ_FLAG_HIDDEN);lv_obj_add_flag(s->label_5,LV_OBJ_FL
AG_HIDDEN);lv_led_on(s->led_3);lv_led_off(s->led_2);}
}

/* ---- param refresh (only widget ptrs, no callback binding) ---- */
static void param_refresh(void) {
    screen_t *s=screen_get(&ui_manager);
    if(!scr_ok()){g_param_valid=false;return;}
    if(g_param_valid&&valid(g_param[0].textarea))return;

    /* g_param[i] = {label, textarea, write_btn, read_btn} for SPxx.
     * The 4 widgets are those visually next to each other on screen.
     * btn_write/read user_data must equal 'i' for modbus_param_to_reg(i). */
    g_param[0]
=(param_row_t){s->label_param_01,s->textarea_2,       s->btn_write_01,s->btn_read_01};
    g_param[1]
=(param_row_t){s->label_param_02,s->textarea_param_02,s->btn_write_02,s->btn_read_02};
    g_param[2]
=(param_row_t){s->label_param_03,s->textarea_param_03,s->btn_write_03,s->btn_read_03};
    g_param[3]
=(param_row_t){s->label_param_04,s->textarea_param_04,s->btn_write_04,s->btn_read_04};
    g_param[4]
=(param_row_t){s->label_param_05,s->textarea_param_05,s->btn_write_05,s->btn_read_05};
    g_param[5]
=(param_row_t){s->label_param_06,s->textarea_param_06,s->btn_write_06,s->btn_read_06};
/* btn07→SP06, btn08→SP07, ..., btn19→SP18.
  * g_param[idx].textarea label and button ARE physically at the same row. */
    g_param[6]
=(param_row_t){s->label_param_15,s->textarea_param_15,s->btn_write_07,s->btn_read_07};
    g_param[7]
=(param_row_t){s->label_param_16,s->textarea_param_16,s->btn_write_08,s->btn_read_08};
    g_param[8]
=(param_row_t){s->label_param_17,s->textarea_param_17,s->btn_write_09,s->btn_read_09};
    g_param[9]
=(param_row_t){s->label_param_18,s->textarea_param_18,s->btn_write_10,s->btn_read_10};
    g_param[10]=(param_row_t){s->label_param_19,s->textarea_param_19,s->btn_write_11,s->btn_read
_11};
    g_param[11]=(param_row_t){s->label_param_07,s->textarea_param_07,s->btn_write_12,s->btn_read
_12};
```

```
    g_param[12]=(param_row_t){s->label_param_08,s->textarea_param_08,s->btn_write_13,s->btn_read
_13};
    g_param[13]=(param_row_t){s->label_param_09,s->textarea_param_09,s->btn_write_14,s->btn_read
_14};
    g_param[14]=(param_row_t){s->label_param_10,s->textarea_param_10,s->btn_write_15,s->btn_read
_15};
    g_param[15]=(param_row_t){s->label_param_11,s->textarea_3,       s->btn_write_16,s->btn_read_
16};
    g_param[16]=(param_row_t){s->label_param_12,s->textarea_param_12,s->btn_write_17,s->btn_read
_17};
    g_param[17]=(param_row_t){s->label_param_13,s->textarea_param_13,s->btn_write_18,s->btn_read
_18};
    g_param[18]=(param_row_t){s->label_param_14,s->textarea_param_14,s->btn_write_19,s->btn_read
_19};

    for(int i=0;i<19;i++)lv_textarea_set_one_line(g_param[i].textarea,true);
    g_param_valid=true;
}

static void save_text(char *dst, int dst_len, lv_obj_t *ta)
{
    if (!dst || dst_len <= 0)
        return;

    if (valid(ta))
        get_ta(ta, dst, dst_len);
}

static void restore_text(lv_obj_t *ta, const char *text)
{
    if (valid(ta) && text)
        lv_textarea_set_text(ta, text);
}

static uint16_t dropdown_get(lv_obj_t *dd)
{
    if (!valid(dd))
        return 0;
    return (uint16_t)lv_dropdown_get_selected(dd);
}

static void dropdown_restore(lv_obj_t *dd, uint16_t selected)
{
    if (valid(dd))
        lv_dropdown_set_selected(dd, selected);
}

static void main_state_save(void)
{
    screen_t *s = screen_get(&ui_manager);

    if (!scr_ok())
        return;

    param_refresh();
    save_text(g_main_state.slave, sizeof(g_main_state.slave), s->textarea_1);
    save_text(g_main_state.tcp_ip, sizeof(g_main_state.tcp_ip), s->textarea_4);
    save_text(g_main_state.tcp_port, sizeof(g_main_state.tcp_port), s->textarea_6);
    save_text(g_main_state.send_log, sizeof(g_main_state.send_log), s->textarea_send);
    save_text(g_main_state.recv_log, sizeof(g_main_state.recv_log), s->textarea_recv);

    for (int i = 0; i < 19; i++) {
        if (g_param_valid)
```

```
            save_text(g_main_state.param[i], sizeof(g_main_state.param[i]), g_param[i].textarea);
        }

        g_main_state.dropdown_com = dropdown_get(s->dropdown_com);
        g_main_state.dropdown_tcp_mode = dropdown_get(s->dropdown_1);
        g_main_state.dropdown_2 = dropdown_get(s->dropdown_2);
        g_main_state.dropdown_baud = dropdown_get(s->dropdown_3);
        g_main_state.dropdown_parity = dropdown_get(s->dropdown_5);
        g_main_state.dropdown_data_bits = dropdown_get(s->dropdown_6);
        g_main_state.dropdown_stop_bits = dropdown_get(s->dropdown_7);
        g_main_state.dropdown_8 = dropdown_get(s->dropdown_8);
        g_main_state.dropdown_crc_mode = dropdown_get(s->dropdown_10);
        g_main_state.valid = true;
    }

    static void main_state_restore(void)
    {
        screen_t *s = screen_get(&ui_manager);

        if (!g_main_state.valid || !s || !s->obj)
            return;

        restore_text(s->textarea_1, g_main_state.slave);
        restore_text(s->textarea_4, g_main_state.tcp_ip);
        restore_text(s->textarea_6, g_main_state.tcp_port);
        restore_text(s->textarea_send, g_main_state.send_log);
        restore_text(s->textarea_recv, g_main_state.recv_log);

        param_refresh();
        for (int i = 0; i < 19; i++) {
            if (g_param_valid)
                restore_text(g_param[i].textarea, g_main_state.param[i]);
        }

        dropdown_restore(s->dropdown_com, g_main_state.dropdown_com);
        dropdown_restore(s->dropdown_1, g_main_state.dropdown_tcp_mode);
        dropdown_restore(s->dropdown_2, g_main_state.dropdown_2);
        dropdown_restore(s->dropdown_3, g_main_state.dropdown_baud);
        dropdown_restore(s->dropdown_5, g_main_state.dropdown_parity);
        dropdown_restore(s->dropdown_6, g_main_state.dropdown_data_bits);
        dropdown_restore(s->dropdown_7, g_main_state.dropdown_stop_bits);
        dropdown_restore(s->dropdown_8, g_main_state.dropdown_8);
        dropdown_restore(s->dropdown_10, g_main_state.dropdown_crc_mode);

        cache_slave_from_textarea(s->textarea_1);
    }

    static bool screen1_active(void)
    {
        screen_1_t *s = screen_1_get(&ui_manager);

        return !g_screen1_leaving && s && valid(s->obj) && lv_scr_act() == s->obj;
    }

    static int screen1_clamp_int(int v, int lo, int hi)
    {
        if (v < lo) return lo;
        if (v > hi) return hi;
        return v;
    }

    static uint32_t screen1_color_from_sel(uint16_t sel)
    {
```

```
    static const uint32_t palette[] = {
        0xff3b30, 0x007aff, 0x34c759, 0xff9500, 0xaf52de,
        0x00c7be, 0xffcc00, 0x8e8e93, 0x5ac8fa, 0xff2d55
    };

    return palette[sel % (sizeof(palette) / sizeof(palette[0]))];
}

static void screen1_channel_default(screen1_channel_state_t *ch, int idx)
{
    static const char *names[] = {"通道一", "通道二", "通道三"};
    static const uint16_t source_sel[] = {0, 0, 0};
    static const uint16_t unit_sel[] = {0, 0, 0};
    static const uint16_t color_sel[] = {0, 1, 2};

    if (!ch)
        return;

    memset(ch, 0, sizeof(*ch));
    ch->enabled = true;
    strncpy(ch->name, names[idx % 3], sizeof(ch->name) - 1);
    ch->source_type = (int)source_sel[idx % 3];
    ch->source_id = 0;
    ch->decimal = 0;
    ch->min_v = 0;
    ch->max_v = 100;
    ch->source_sel = source_sel[idx % 3];
    ch->unit_sel = unit_sel[idx % 3];
    ch->color_sel = color_sel[idx % 3];
    ch->raw_min = 0xffffffffu;
    ch->raw_max = 0;
}
```